package-lock.json

 file is an automatically generated file in a Node.js project that is created when you run `npm install`. It ensures that the exact versions of dependencies and their sub-dependencies are installed, providing a consistent and reproducible environment for your project. This file locks the versions of the packages, preventing unintended updates that could potentially break the application.

The

package-lock.json

 file contains detailed metadata about each dependency, including the version, resolved URL, integrity hash, and any nested dependencies. This information helps in maintaining the integrity and security of the project by ensuring that the same versions of dependencies are used across different environments and installations.

In summary, the

package-lock.json

 file is crucial for maintaining consistency and reliability in a Node.js project by locking down the versions of all installed packages and their dependencies.

The provided

tsconfig.json

 file is a configuration file for a TypeScript project. It specifies various compiler options and settings that control how the TypeScript compiler (`tsc`) processes the project's source code.

The `compilerOptions` section contains several key settings:
- `"target": "es5"`: This setting specifies that the TypeScript code should be transpiled to ECMAScript 5, ensuring compatibility with older JavaScript environments.
- `"lib": ["dom", "dom.iterable", "esnext"]`: This array specifies the libraries to include in the compilation. It includes the DOM library for browser APIs, the DOM iterable library for iterable objects, and the latest ECMAScript features.
- `"noImplicitThis": false`: This option allows the use of `this` in a way that might be implicitly `any`.
- `"allowJs": true`: This setting allows JavaScript files to be compiled alongside TypeScript files.

- `"skipLibCheck": true`: This option skips type checking of declaration files (`.d.ts`), which can speed up the compilation process.
- `"esModuleInterop": true` and `"allowSyntheticDefaultImports": true`: These settings enable interoperability between CommonJS and ES modules, allowing default imports from modules that do not have a default export.
- `"strict": true`: This enables all strict type-checking options, improving code quality and reducing potential runtime errors.
- `"forceConsistentCasingInFileNames": true`: This ensures that file names are treated with consistent casing, preventing issues on case-sensitive file systems.
- `"noFallthroughCasesInSwitch": true`: This prevents fallthrough cases in switch statements, which can lead to bugs.
- `"module": "esnext"` and `"moduleResolution": "node"`: These settings specify the module code generation and module resolution strategy, respectively, using the latest ECMAScript module syntax and Node.js module resolution.
- `"resolveJsonModule": true`: This allows importing JSON files as modules.
- `"isolatedModules": true`: This ensures that each file is treated as a separate module, which is useful for certain build tools and environments.
- `"noEmit": true`: This prevents the compiler from emitting output files, useful when TypeScript is used for type checking only.
- `"jsx": "react-jsx"`: This setting specifies the JSX code generation, targeting the React JSX runtime.

The `include` section specifies the files to include in the compilation. In this case, it includes all files in the `src` directory, ensuring that only the source code is compiled. This configuration file helps maintain consistent and predictable behavior across different environments and development setups.

The

package.json

file is a crucial part of any Node.js project. It contains metadata about the project, including its dependencies, scripts, and other configurations. Here's a breakdown of the key sections in the provided

package.json

file:

Metadata
- `"name": "frontend"`: The name of the project.
- `"version": "0.1.0"`: The current version of the project.

- `"private": true`: This prevents the project from being accidentally published to the npm registry.

Dependencies
The `dependencies` section lists the packages required for the project to run:
```json
"dependencies": {
  "@types/jest": "^27.5.2",
  "@types/node": "^16.18.11",
  "@types/react": "^18.0.26",
  "@types/react-dom": "^18.0.10",
  "axios": "^1.2.1",
  "bootstrap": "^5.2.3",
  "mdb-react-ui-kit": "^5.1.0",
  "nodemailer": "^6.9.16",
  "react": "^18.2.0",
  "react-bootstrap": "^2.7.0",
  "react-dom": "^18.2.0",
  "react-scripts": "^5.0.1"
}
```
These packages are necessary for the application to function correctly.

Scripts
The `scripts` section defines commands that can be run using `npm run <script-name>`:
```json
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "eject": "react-scripts eject"
}
```
- `"start"`: Runs the development server.
- `"build"`: Builds the project for production.
- `"eject"`: Ejects the configuration files from `react-scripts` for customization.

ESLint Configuration
The `eslintConfig` section configures ESLint, a tool for identifying and fixing problems in JavaScript code:
```json
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
```

```
 ]
}
```
This extends the default ESLint configurations for React applications and Jest testing.

Browserslist
The `browserslist` section specifies the target browsers for the project:
```json
"browserslist": {
 "production": [
  ">0.2%",
  "not dead",
  "not op_mini all"
 ],
 "development": [
  "last 1 chrome version",
  "last 1 firefox version",
  "last 1 safari version"
 ]
}
```
This helps in configuring tools like Babel and Autoprefixer to support the specified browsers.

Development Dependencies
The `devDependencies` section lists packages needed only during development:
```json
"devDependencies": {
 "cross-env": "^7.0.3",
 "typescript": "^4.9.5"
}
```
- `"cross-env"`: Allows setting environment variables across different platforms.
- `"typescript"`: The TypeScript compiler.

This

package.json

 file provides a comprehensive setup for a React project with TypeScript, including necessary dependencies, scripts, and configurations for development and production environments.

Similar code found with 2 license types

The

package-lock.json

file is an automatically generated file that is created when you run `npm install` in a Node.js project. It ensures that the exact versions of dependencies and their sub-dependencies are installed, providing a consistent and reproducible environment for your project. Here is a breakdown of the key sections in the provided

package-lock.json

file:

Metadata
- `"name": "frontend"` : The name of the project.
- `"version": "0.1.0"` : The current version of the project.
- `"lockfileVersion": 3` : The version of the lockfile format. This ensures compatibility with different versions of npm.
- `"requires": true` : Indicates that the project has dependencies that need to be installed.

Packages
The `packages` section lists all the packages and their specific versions that are installed in the project. It includes both direct dependencies and nested dependencies.

Root Package
The root package (your project) is represented by an empty string key `""` :
```json
"packages": {
 "": {
  "name": "frontend",
  "version": "0.1.0",
  "dependencies": {
   "@types/jest": "^27.5.2",
   "@types/node": "^16.18.11",
   "@types/react": "^18.0.26",
   "@types/react-dom": "^18.0.10",
   "axios": "^1.2.1",
   "bootstrap": "^5.2.3",
   "mdb-react-ui-kit": "^5.1.0",
   "nodemailer": "^6.9.16",
   "react": "^18.2.0",
   "react-bootstrap": "^2.7.0",
   "react-dom": "^18.2.0",
```

```
    "react-scripts": "^5.0.1"
   },
   "devDependencies": {
    "cross-env": "^7.0.3",
    "typescript": "^4.9.5"
   }
  }
}
```
This section lists the dependencies and devDependencies as specified in your `package.json`.

Nested Dependencies
Each dependency listed in the `packages` section has its own entry with detailed information:
```json
"node_modules/@ampproject/remapping": {
 "version": "2.2.0",
 "resolved": "https://registry.npmjs.org/@ampproject/remapping/-/remapping-2.2.0.tgz",
 "integrity": "sha512-
qRmjj8nj9qmLTQXXmaR1cck3UXSRMPrbsLJAasZpF+t3riI71BXed5ebIOYwQntykeZuhjsdweEc9BxH5Jc26w==",
 "dependencies": {
  "@jridgewell/gen-mapping": "^0.1.0",
  "@jridgewell/trace-mapping": "^0.3.9"
 },
 "engines": {
  "node": ">=6.0.0"
 }
}
```
- `"version"`: The specific version of the package.
- `"resolved"`: The URL from which the package was downloaded.
- `"integrity"`: A hash to ensure the package has not been tampered with.
- `"dependencies"`: Lists any dependencies that this package requires.
- `"engines"`: Specifies the Node.js version compatibility.

Example of Another Dependency
```json
"node_modules/@babel/code-frame": {
 "version": "7.26.2",
 "resolved": "https://registry.npmjs.org/@babel/code-frame/-/code-frame-7.26.2.tgz",
```

"integrity": "sha512-
qRmjj8nj9qmLTQXXmaR1cck3UXSRMPrbsLJAasZpF+t3riI71BXed5ebIOYwQntykeZuhjsdwe
Ec9BxH5Jc26w=="
}
```
This entry provides similar details for the `@babel/code-frame` package.

### Summary
The

package-lock.json

file ensures that your project installs the exact same versions of dependencies and their sub-dependencies every time, providing consistency and reliability. It includes detailed metadata about each package, such as version, resolved URL, integrity hash, and any nested dependencies. This file is crucial for maintaining the integrity and security of your project by locking down the versions of all installed packages.

Similar code found with 2 license types

The

AxiosResponse

type is part of the Axios library, which is a popular HTTP client for making requests to APIs in JavaScript and TypeScript. It provides a way to handle the response from an HTTP request.

### Importing Axios and AxiosResponse
First, ensure you have Axios installed in your project:
```bash
npm install axios
```

Then, you can import Axios and the

AxiosResponse

type in your TypeScript file:
```typescript
import axios, { AxiosResponse } from 'axios';
```

Using AxiosResponse
```

The

AxiosResponse

 type represents the structure of the response object returned by Axios. Here is a breakdown of its properties:
- `data`: The response data from the server.
- `status`: The HTTP status code from the server response.
- `statusText`: The HTTP status text from the server response.
- `headers`: The headers sent back by the server.
- `config`: The configuration object used for the request.
- `request`: The request object.

Example Usage
Here is an example of how to use

AxiosResponse

 in a TypeScript function to make a GET request and handle the response:
```typescript
import axios, { AxiosResponse } from 'axios';

interface User {
  id: number;
  name: string;
  email: string;
}

async function fetchUsers(): Promise<User[]> {
  try {
    const response: AxiosResponse<User[]> = await
axios.get('https://api.example.com/users');
    console.log('Status:', response.status);
    console.log('Data:', response.data);
    return response.data;
  } catch (error) {
    console.error('Error fetching users:', error);
    throw error;
  }
}

fetchUsers().then(users => {
  users.forEach(user => {
    console.log(`User: ${user.name}, Email: ${user.email}`);
```

```
  });
});
```

Explanation
1. **Importing Axios and AxiosResponse**: The `axios` module and

AxiosResponse

 type are imported.
2. **Defining the User Interface**: An interface `User` is defined to type the response data.
3. **fetchUsers Function**:
   - The function `fetchUsers` makes an asynchronous GET request to fetch users.
   - The response is typed as

AxiosResponse<User[]>

, indicating that the response data is an array of `User` objects.
   - The function logs the status and data from the response and returns the data.
4. **Handling the Response**: The `fetchUsers` function is called, and the returned users are logged to the console.

This example demonstrates how to use the

AxiosResponse

 type to handle HTTP responses in a TypeScript project, ensuring type safety and better code readability.

The provided code is a TypeScript file that defines a class for handling tasks, including making HTTP requests to a server to fetch tasks. Here's a detailed breakdown of the code:

Imports
```typescript
import axios, { AxiosResponse } from "axios";
import { config } from "./config";
```
- **axios**: A popular HTTP client for making requests.
- **AxiosResponse**: A type that represents the structure of the response object returned by Axios.
- **config**: An imported configuration object that likely contains the server address.

Interfaces and Enums
ITask Interface
```typescript
export interface ITask {
  id?: number;
  title: string;
  description: string;
  state: string;
  listID: number;
}
```
This interface describes the structure of a task object:
-

id

 (optional): The unique identifier for the task.
-

title

: The title of the task.
-

description

: A description of the task.
-

state

: The current state of the task.
-

listID

: The ID of the list to which the task belongs.

ETaskStates Enum
```typescript
export enum ETaskStates {
  PorComecar,
  Acabada,
```

```
    EmCurso
}
```
This enumeration represents the different states a task can have:
-

PorComecar

: Task has not started.
-

Acabada

: Task is finished.
-

EmCurso

: Task is in progress.

Worker Class
The

Worker

 class contains methods for interacting with tasks.

getTasks Method
```typescript
public async getTasks(listId: number): Promise<ITask[]> {
    console.log("getTasks");
    const response: AxiosResponse = await axios.get(`${config.serverAddress}/getTasks/` +
listId);
    return response.data;
}
```
- **Purpose**: Fetches tasks from the server based on the list ID.
- **Parameters**:

listId

 - The ID of the list for which tasks are to be fetched.
- **Returns**: A promise that resolves to an array of

ITask

 objects.

Explanation
1. **Imports**: The code imports necessary modules and types, including Axios for making HTTP requests and a configuration object for server details.
2. **ITask Interface**: Defines the structure of a task, ensuring type safety when working with task objects.
3. **ETaskStates Enum**: Provides a set of predefined states for tasks, improving code readability and reducing errors.
4. **Worker Class**: Contains methods for task-related operations.
  - **getTasks Method**: Makes an asynchronous GET request to fetch tasks for a specific list. It logs a message to the console, makes the request using Axios, and returns the response data.

Example Usage
Here is an example of how you might use the

Worker

 class to fetch tasks:
```typescript
const worker = new Worker();
worker.getTasks(1).then(tasks => {
  tasks.forEach(task => {
    console.log(`Task: ${task.title}, State: ${task.state}`);
  });
}).catch(error => {
  console.error('Error fetching tasks:', error);
});
```
This example creates an instance of the

Worker

 class, calls the

getTasks

 method with a list ID of `1`, and logs each task's title and state to the console. It also handles any errors that might occur during the request.

This code provides a clear and structured way to manage tasks, including fetching them from a server, with TypeScript ensuring type safety and better code maintainability.

The provided code defines a function

createState

that initializes and returns the state object for a component, likely a React component. This state object includes various properties and methods to manage the application's state, such as the current view, user information, tasks, categories, and lists. Here's a detailed breakdown of the code:

Function Definition
```typescript
export default function createState(inParentComponent: BaseLayout) {
  return {
    // State properties and methods
  };
}
```
- **createState**: A function that takes an

inParentComponent

parameter of type

BaseLayout

and returns an object representing the state.

State Properties
- **currentView**: The view currently being displayed (e.g., "welcome", "login", "register", "logged").
- **error**: A string representing the current error type (e.g., "notEqualPassword", "emptyPassword").
- **isLoggedIn**: A boolean indicating whether the user is logged in.
- **showModal**: A boolean indicating whether a modal should be shown.
- **modalView**: A string indicating which modal view to show (e.g., "createTask", "createCategory").
- **types**: An array of category types (e.g., "Universidade", "Pessoal").
- **currentTask**: An object representing the currently selected task.
- **currentList**: An object representing the currently selected list.
- **currentCatID**: The ID of the last opened category.

- **tasks**: An array of tasks.
- **categories**: An array of categories.
- **all_lists**: An array of all lists.
- **user**: An object representing the user's data.

State Methods
- **changeCurrentProjectId**: Updates the current project ID.
- **registerUser**: Registers a new user.
- **loginUser**: Authenticates a user.
- **addTask**: Adds a new task.
- **addCategory**: Adds a new category.
- **addList**: Adds a new list.
- **updateListView**: Updates the list view with tasks and list details.
- **selectTask**: Selects a task and shows the task detail view.
- **deleteTask**: Deletes a task.
- **updateTaskState**: Updates the state of a task.
- **deleteList**: Deletes a list.
- **deleteCategory**: Deletes a category.
- **deleteAccount**: Deletes the user's account.
- **logoutUser**: Logs out the user.
- **showView**: Changes the current view.
- **updateCat**: Updates the current category ID.
- **showModalView**: Toggles the modal view and updates the modal view and category ID.
- **updateUserInfo**: Updates the user's information.
- **updateError**: Updates the error state.

Example Usage
Here is an example of how you might use the

createState

 function in a React component:
```typescript
import React, { Component } from 'react';
import createState from './state';

class BaseLayout extends Component {
  constructor(props) {
    super(props);
    this.state = createState(this);
  }

  render() {
```

```
    return (
      <div>
        {/* Render your component based on the state */}
      </div>
    );
  }
}

export default BaseLayout;
```
This example demonstrates how to initialize the state using the

createState

 function and use it within a React component.

Summary
The

createState

 function provides a comprehensive state management solution for a React component, including properties and methods to handle user authentication, task management, category management, and view updates. This approach ensures a structured and maintainable way to manage the application's state.

The provided code defines a

Worker

 class within a TypeScript file, likely named

List.ts

, which handles various operations related to lists. This class uses Axios to make HTTP requests to a server. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```typescript
import axios, { AxiosResponse } from 'axios';
```

```
import { config } from './config';
import { IList } from './interfaces';
```
- **axios**: A popular HTTP client for making requests.
- **AxiosResponse**: A type that represents the structure of the response object returned by Axios.
- **config**: An imported configuration object that likely contains the server address.
- **IList**: An interface representing the structure of a list object.

Worker Class
The

Worker

 class contains methods for interacting with lists.

getLists Method
```typescript
public async getLists(userID: number): Promise<IList[]> {
   console.log("getLists");
   const response: AxiosResponse = await axios.get(`${config.serverAddress}/getLists/` + userID);
   return response.data;
}
```
- **Purpose**: Fetches lists from the server based on the user ID.
- **Parameters**:

userID

 - The ID of the user whose lists are to be fetched.
- **Returns**: A promise that resolves to an array of

IList

 objects.

getList Method
```typescript
public async getList(id: number): Promise<IList> {
   console.log("getList");
   const response: AxiosResponse = await axios.get(`${config.serverAddress}/getList/` + id);
   return response.data;
```

}
```
- **Purpose**: Fetches a specific list from the server based on the list ID.
- **Parameters**:

id

 - The ID of the list to be fetched.
- **Returns**: A promise that resolves to an

IList

 object.

#### addList Method
```typescript
public async addList(list: IList): Promise<IList> {
   console.log("addList");
   const response: AxiosResponse = await
axios.post(`${config.serverAddress}/createList/`, list);
   return response.data;
}
```
- **Purpose**: Adds a new list to the server.
- **Parameters**:

list

 - The list object to be added.
- **Returns**: A promise that resolves to the added

IList

 object.

deleteList Method
```typescript
public async deleteList(listID: number): Promise<boolean> {
   console.log("deleteList");
   const response: AxiosResponse = await
axios.delete(`${config.serverAddress}/deleteList/` + listID);
   return response.data;
}
```
```

- **Purpose**: Deletes a list from the server based on the list ID.
- **Parameters**:

listID

 - The ID of the list to be deleted.
- **Returns**: A promise that resolves to a boolean indicating the success of the deletion.

Example Usage
Here is an example of how you might use the

Worker

 class to manage lists:
```typescript
const listWorker = new Worker();

// Fetch lists for a user
listWorker.getLists(1).then(lists => {
  lists.forEach(list => {
    console.log(`List: ${list.title}`);
  });
}).catch(error => {
  console.error('Error fetching lists:', error);
});

// Add a new list
const newList: IList = { title: 'New List', description: 'Description', userID: 1, categoryID: 1 };
listWorker.addList(newList).then(addedList => {
  console.log('Added List:', addedList);
}).catch(error => {
  console.error('Error adding list:', error);
});

// Delete a list
listWorker.deleteList(1).then(success => {
  if (success) {
    console.log('List deleted successfully');
  } else {
    console.error('Error deleting list');
  }
}).catch(error => {
  console.error('Error deleting list:', error);
});
```

```
This example demonstrates how to:
- Fetch lists for a user.
- Add a new list.
- Delete a list.

### Summary
The

Worker

 class provides methods for managing lists, including fetching, adding, and deleting lists. It uses Axios to make HTTP requests to a server, ensuring type safety and better code maintainability with TypeScript. This approach provides a clear and structured way to handle list-related operations in your application.



The provided code is the entry point for a React application, written in TypeScript and JSX (TSX). It sets up the React application, imports necessary modules, and renders the main component into the DOM. Here's a detailed breakdown of the code:

### Imports
```tsx
// Importações do React.
import React from "react";
import ReactDOM from "react-dom/client";

// Importação do ficheiro de estilos CSS.
import './assets/styles/index.css';

// Importação do componente BaseLayout.
import BaseLayout from './components/BaseLayout';
```
- **React**: The core React library for building user interfaces.
- **ReactDOM**: The React library for rendering components to the DOM.
- **index.css**: A CSS file for global styles.
- **BaseLayout**: The main component of the application, imported from the `components` directory.

Creating the Root
```tsx
// Criação da raiz do React no elemento com o ID "root".
const root = ReactDOM.createRoot(document.getElementById("root") as HTMLElement);
```
```

```
- **ReactDOM.createRoot**: Creates a root for the React application. This is a new method introduced in React 18 for concurrent rendering.
- **document.getElementById("root")**: Selects the DOM element with the ID

root

 where the React application will be mounted.
- **as HTMLElement**: Type assertion to ensure TypeScript knows the selected element is an HTML element.

### Rendering the Application
```tsx
// Renderização do conteúdo na raiz.
root.render(
  <React.StrictMode>
    {/* Div principal da aplicação com um fundo de cor #FFF6F6 */}
    <div className={"App"} style={{backgroundColor: "#FFF6F6"}}>
      {/* Renderização do componente BaseLayout */}
      <BaseLayout/>
    </div>
  </React.StrictMode>
);
```
- **React.StrictMode**: A wrapper component that helps identify potential problems in the application. It activates additional checks and warnings for its descendants.
- **div with className "App"**: The main container div for the application, styled with a background color of `#FFF6F6`.
- **BaseLayout**: The main component of the application, rendered inside the main container div.

Summary
This code sets up the entry point for a React application. It imports necessary modules, creates a root for the React application, and renders the main

BaseLayout

 component inside a styled div. The use of

React.StrictMode

 helps ensure that the application follows best practices and identifies potential issues during development.
```

Example of index.html
For this code to work, you need an `index.html` file with a

root

 element. Here is an example:
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>React App</title>
  <link rel="stylesheet" href="assets/styles/index.css">
</head>
<body>
  <div id="root"></div>
  <script src="index.tsx"></script>
</body>
</html>
```
This `index.html` file includes a

div

 with the ID

root

, which is where the React application will be mounted. It also links to the CSS file and includes the script for the TypeScript entry point.

This setup provides a clear and structured way to initialize and render a React application using TypeScript, ensuring type safety and better code maintainability.

Similar code found with 1 license type

The provided code snippet defines a configuration object in TypeScript, which includes the server address for your application. Here's a detailed breakdown of the code:

Configuration Object
```typescript
const config: { serverAddress: string } = {
```

```
  serverAddress: "http://localhost:8080" // Endereço do servidor
};
```
- **config**: A constant variable that holds the configuration object.
- **{ serverAddress: string }**: The type annotation for the

config

object, specifying that it should have a property

serverAddress

of type `string`.
- **serverAddress**: The actual server address, set to `"http://localhost:8080"`, which is likely the address of your development server.

Usage
This configuration object can be used throughout your application to make HTTP requests to the specified server. By centralizing the server address in a configuration file, you can easily update it without having to change multiple parts of your codebase.

Example Usage in an Axios Request
Here is an example of how you might use the

config

object in an Axios request:
```typescript
import axios from 'axios';
import { config } from './config';

async function fetchData() {
  try {
    const response = await axios.get(`${config.serverAddress}/api/data`);
    console.log('Data:', response.data);
  } catch (error) {
    console.error('Error fetching data:', error);
  }
}

fetchData();
```
In this example:
- **Importing config**: The

config

 object is imported from the

config

 file.
- **Using config.serverAddress**: The

serverAddress

 property is used to construct the URL for the Axios GET request.

Summary
The

config

 object provides a centralized way to manage configuration settings, such as the server address, in your application. This approach improves maintainability and makes it easier to update configuration settings without modifying multiple parts of your codebase.

Full Example of

config.ts

Here is a complete example of what your

config.ts

 file might look like:
```typescript

export const config: { serverAddress: string } = {
  serverAddress: "http://localhost:8080" // Endereço do servidor
};
```
This file exports the

config

object, making it available for import in other parts of your application.

The provided code defines a

Worker

class within a TypeScript file, likely named

Categories.ts

, which handles various operations related to categories. This class uses Axios to make HTTP requests to a server. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```typescript
import axios, { AxiosResponse } from 'axios';
import { config } from './config';
import { ICategory } from './interfaces';
```
- **axios**: A popular HTTP client for making requests.
- **AxiosResponse**: A type that represents the structure of the response object returned by Axios.
- **config**: An imported configuration object that likely contains the server address.
- **ICategory**: An interface representing the structure of a category object.

Worker Class
The

Worker

class contains methods for interacting with categories.

addNewCategory Method
```typescript
public async addNewCategory(cat: ICategory): Promise<ICategory> {
  console.log("addCategory");
  const response: AxiosResponse = await
axios.post(`${config.serverAddress}/createCategory/`, cat);
  return response.data;
}
```

- **Purpose**: Adds a new category to the server.
- **Parameters**:

cat

 - The category object to be added.
- **Returns**: A promise that resolves to the added

ICategory

 object.

getCategories Method
```typescript
public async getCategories(userID: number): Promise<ICategory[]> {
   console.log("getCategories");
   const response: AxiosResponse = await
axios.get(`${config.serverAddress}/getCategories/` + userID);
   return response.data;
}
```
- **Purpose**: Fetches categories from the server based on the user ID.
- **Parameters**:

userID

 - The ID of the user whose categories are to be fetched.
- **Returns**: A promise that resolves to an array of

ICategory

 objects.

deleteCategory Method
```typescript
public async deleteCategory(id: number): Promise<boolean> {
   console.log("deleteCategory");
   const response: AxiosResponse = await
axios.delete(`${config.serverAddress}/deleteCategory/` + id);
   return response.data;
}
```
- **Purpose**: Deletes a category from the server based on the category ID.
- **Parameters**:

id

 - The ID of the category to be deleted.
- **Returns**: A promise that resolves to a boolean indicating the success of the deletion.

Example Usage
Here is an example of how you might use the

Worker

 class to manage categories:
```typescript
const categoryWorker = new Worker();

// Add a new category
const newCategory: ICategory = { title: 'New Category', description: 'Description', userID: 1
};
categoryWorker.addNewCategory(newCategory).then(addedCategory => {
  console.log('Added Category:', addedCategory);
}).catch(error => {
  console.error('Error adding category:', error);
});

// Fetch categories for a user
categoryWorker.getCategories(1).then(categories => {
  categories.forEach(category => {
    console.log(` Category: ${category.title}` );
  });
}).catch(error => {
  console.error('Error fetching categories:', error);
});

// Delete a category
categoryWorker.deleteCategory(1).then(success => {
  if (success) {
    console.log('Category deleted successfully');
  } else {
    console.error('Error deleting category');
  }
}).catch(error => {
  console.error('Error deleting category:', error);
});
```

This example demonstrates how to:
- Add a new category.
- Fetch categories for a user.
- Delete a category.

Summary
The

Worker

 class provides methods for managing categories, including adding, fetching, and deleting categories. It uses Axios to make HTTP requests to a server, ensuring type safety and better code maintainability with TypeScript. This approach provides a clear and structured way to handle category-related operations in your application.

The provided code defines a simple React functional component named

App

 in a TypeScript file, likely named

App.tsx

. This component serves as the main entry point for the application. Here's a detailed breakdown of the code:

Imports
```tsx
// Importação do ficheiro de estilos CSS específico para a aplicação.
import './assets/styles/App.css';

// Importação do componente BaseLayout.
import BaseLayout from './components/BaseLayout';
```
- **App.css**: A CSS file that contains styles specific to the application.
- **BaseLayout**: The main layout component of the application, imported from the `components` directory.

App Component
```tsx
// Definição do componente funcional App.
function App() {
  return (
```

```
      // Renderização do componente BaseLayout.
      <BaseLayout/>
    );
}
```
- **App**: A functional component that serves as the root component of the application.
- **BaseLayout**: The main layout component is rendered inside the

App

 component.

Export
```tsx
// Exportação do componente App como o padrão.
export default App;
```
- **default export**: The

App

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

App

 component is a simple functional component that imports and applies global styles from

App.css

 and renders the

BaseLayout

 component. This setup provides a clean and organized structure for the main entry point of the application.

Example Usage in index.tsx
Here is an example of how the

App

component might be used in the `index.tsx` file:
```tsx
import React from "react";
import ReactDOM from "react-dom/client";
import './assets/styles/index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById("root") as HTMLElement);

root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```
In this example:
- **App**: The

App

component is imported and rendered inside the `React.StrictMode` wrapper.
- **ReactDOM.createRoot**: Creates a root for the React application and mounts the

App

component to the DOM element with the ID `root`.

This setup ensures that the

App

component is the main entry point of the application, providing a structured and maintainable way to manage the application's layout and styles.

The provided code defines a React functional component named

WelcomeView

in a TypeScript file, likely named

WelcomeView.tsx

. This component serves as a welcome screen for the application, providing options for registration, login, and information about the application. Here's a detailed breakdown of the code:

Imports
```tsx
// Importação do componente ModalPop.
import Modal from "../components/ModalPop";
import { Button } from 'react-bootstrap';
```
- **Modal**: A modal component imported from the `components` directory.
- **Button**: A button component from the `react-bootstrap` library.

WelcomeView Component
```tsx
// Definição do componente funcional WelcomeView.
const WelcomeView = ({ state }: { state: any }) => {
  return (
    // Div principal com classes para centralização e estilo específico.
    <div className="centered-div welcomeview">
      {/* Renderização do componente ModalPop com o estado passado como prop */}
      <Modal state={state} />
      <div>
        {/* Título principal com estilo de texto roxo e tamanho grande */}
        <h1 className="centered-div2 text-purple large-text">Qual é o teu próximo<br />
passo académico?</h1>
        {/* Texto descritivo com estilo de texto roxo e tamanho médio */}
        <div className="centered-div2 text-purple medium-text">
          Organiza o teu mundo universitário<br />
          listas, projetos e lembretes organizados de forma mais produtiva.
        </div>
        {/* Botão para registo com estilo personalizado */}
        <div className="centered-div2">
          <Button style={{ backgroundColor: "purple", borderColor: "purple" }}
className="btn btn-primary mx-3 my-4" onClick={() => state.showView("register")}>Obter
Gratuitamente</Button>
        </div>
        {/* Texto e botão para login com estilo personalizado */}
        <div className="centered-div2 mt-2">
          <span>Já tem conta? </span>
          <Button style={{ backgroundColor: "#FF00FF", borderColor: "purple", color:
"white" }} className="btn btn-secondary mx-3" onClick={() =>
state.showView("login")}>Login</Button>
        </div>
```

```
            {/* Botão para a página "Sobre nós" com estilo transparente */}
            <div className="centered-div2 mt-2">
              <Button style={{ backgroundColor: "transparent", borderColor: "transparent", color: "black" }} className="btn btn-link" onClick={() => state.showView("about")}>Sobre nós</Button>
            </div>
          </div>
        </div>
    );
}
```
- **WelcomeView**: A functional component that takes a

state

 prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Main Div**: The main container div with classes `centered-div` and `welcomeview` for styling and centralization.
- **Modal Component**: The

Modal

 component is rendered with the

state

 prop passed to it.
- **Title**: A heading (

h1

) with the class `centered-div2 text-purple large-text` for styling.
- **Description**: A div with the class `centered-div2 text-purple medium-text` containing descriptive text.
- **Register Button**: A button with custom styles and the class `btn btn-primary mx-3 my-4`. It triggers the

showView

 method from the

state

 prop to navigate to the registration view.
- **Login Section**: A span and a button with custom styles and the class `btn btn-secondary mx-3`. The button triggers the

showView

 method to navigate to the login view.
- **About Us Button**: A button with transparent styles and the class `btn btn-link`. It triggers the

showView

 method to navigate to the "about" view.

Export
```tsx
// Exportação do componente WelcomeView como o padrão.
export default WelcomeView;
```
- **default export**: The

WelcomeView

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

WelcomeView

 component serves as a welcome screen for the application, providing options for registration, login, and information about the application. It uses the

Modal

 component to display additional information and the

Button

component from `react-bootstrap` for styled buttons. The component is styled using CSS classes and inline styles, and it interacts with the application's state through the

state

prop.

Example Usage
Here is an example of how the

WelcomeView

component might be used in a parent component:
```tsx
import React from 'react';
import WelcomeView from './WelcomeView';

const App = () => {
  const state = {
    showView: (view: string) => {
      console.log(`Navigating to ${view}`);
    }
  };

  return (
    <WelcomeView state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with a

showView

method that logs the view to the console.
- **WelcomeView**: The

WelcomeView

component is rendered with the

state

prop passed to it.

This setup provides a clear and structured way to manage the welcome screen of the application, ensuring a user-friendly interface for new and returning users.

The provided code defines a React class component named

RegisterView

in a TypeScript file, likely named

RegisterView.tsx

. This component provides a registration form for users to sign up by entering their name, email, password, and password confirmation. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Form, Button, Alert } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Form, Button, Alert**: Components from the `react-bootstrap` library for building forms, buttons, and alerts.

RegisterView Component
The

RegisterView

component extends

React.Component

and manages its own state.

State Initialization
```tsx
state = {
  name: "", // Nome do utilizador
  email: "", // Email do utilizador
```

```
    password: "", // Palavra-passe do utilizador
    passwordConfirm: "" // Confirmação da palavra-passe
};
```
- **state**: An object that holds the initial state of the component, including

name

,

email

,

password

, and

passwordConfirm

.

handleChange Method
```tsx
handleChange = (e: any) => {
    this.setState({ [e.target.id]: e.target.value });
}
```
- **handleChange**: A method that updates the state when the user types into the input fields. It uses the

id

 attribute of the input elements to determine which state property to update.

validEmail Method
```tsx
validEmail(email: string) {
    return email.indexOf("@");
}
```
- **validEmail**: A method that checks if the email contains an "@" symbol, returning its index. If the index is `-1`, the email is considered invalid.

handleSubmit Method
```tsx
handleSubmit = () => {
  // Verifica se o nome ou email estão vazios.
  if (this.state.name === "" || this.state.email === "") {
    this.props.state.updateError("missingData");
    return;
  }
  // Verifica se a palavra-passe ou a confirmação da palavra-passe estão vazias.
  if (this.state.password === "" || this.state.passwordConfirm === "") {
    this.props.state.updateError("emptyPassword");
    return;
  }
  // Verifica se a palavra-passe e a confirmação da palavra-passe são iguais.
  if (this.state.password !== this.state.passwordConfirm) {
    this.props.state.updateError("notEqualPassword");
    return;
  }
  // Verifica se o email é válido.
  if (this.validEmail(this.state.email) === -1) {
    this.props.state.updateError("incorrectEmail");
    return;
  }

  // Regista o utilizador com os dados fornecidos.
  this.props.state.registerUser({
    name: this.state.name,
    email: this.state.email,
    password: this.state.password
  });
}
```
- **handleSubmit**: A method that handles form submission. It performs various checks to ensure the form data is valid and then calls the

registerUser

 method from the

state

 prop to register the user.

render Method

```tsx
render() {
  return (
    <>
      {/* Div principal com classe para estilo específico */}
      <div className="register-container">
        {/* Título principal com estilo de texto preto e negrito */}
        <h1 className="text-black fw-bolder">Registar</h1>
        {/* Formulário de registo */}
        <Form>
          {/* Grupo de formulário para o campo Nome */}
          <Form.Group className="mb-3">
            <Form.Label>Nome</Form.Label>
            <Form.Control
              type="text"
              id="name"
              value={this.state.name}
              onChange={this.handleChange}
            />
          </Form.Group>
          {/* Grupo de formulário para o campo Email */}
          <Form.Group className="mb-3">
            <Form.Label>Email</Form.Label>
            <Form.Control
              type="email"
              id="email"
              value={this.state.email}
              onChange={this.handleChange}
            />
          </Form.Group>
          {/* Grupo de formulário para o campo Palavra-Passe */}
          <Form.Group className="mb-3">
            <Form.Label>Palavra-Passe</Form.Label>
            <Form.Control
              type="password"
              id="password"
              value={this.state.password}
              onChange={this.handleChange}
            />
          </Form.Group>
          {/* Grupo de formulário para o campo Confirmar Palavra-Passe */}
          <Form.Group className="mb-3">
            <Form.Label>Confirmar Palavra-Passe</Form.Label>
            <Form.Control
```

```
                    type="password"
                    id="passwordConfirm"
                    value={this.state.passwordConfirm}
                    onChange={this.handleChange}
                  />
                </Form.Group>
                {/* Botão para submeter o formulário de registo */}
                <Button className="btn btn-primary"
onClick={this.handleSubmit}>Registar</Button>
            </Form>
            {/* Exibe um alerta se houver um erro no estado */}
            {this.props.state.error && (
              <Alert variant="danger" className="mt-3">
                {this.props.state.error}
              </Alert>
            )}
          </div>
        </>
    );
}
```
- **render**: The method that returns the JSX to render the component. It includes a form with input fields for name, email, password, and password confirmation, and a submit button. If there is an error in the state, it displays an alert with the error message.

Summary
The

RegisterView

 component provides a registration form for users to sign up by entering their name, email, password, and password confirmation. It includes methods to handle input changes, validate the email format, and handle form submission. The component uses Bootstrap components for styling and layout.

Example Usage
Here is an example of how the

RegisterView

 component might be used in a parent component:
```tsx
import React from 'react';
import RegisterView from './RegisterView';
```

```
const App = () => {
  const state = {
    error: null,
    updateError: (error: string) => {
      console.log(`Error: ${error}`);
    },
    registerUser: (user: { name: string, email: string, password: string }) => {
      console.log('Registering user:', user);
    }
  };

  return (
    <RegisterView state={state} />
  );
};

export default App;
```

In this example:
- **state**: An object with

error

,

updateError

, and

registerUser

 methods to manage the application's state.
- **RegisterView**: The

RegisterView

 component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to manage user registration in the application, ensuring a user-friendly interface for new users.

The provided code defines a React functional component named

ProjectView

in a TypeScript file, likely named

ProjectView.tsx

. This component renders a header and a main section styled with classes and inline styles. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { MDBContainer, MDBRow, MDBCol } from 'mdb-react-ui-kit';
import Header from './Header';
```
- **React**: The core React library for building user interfaces.
- **MDBContainer, MDBRow, MDBCol**: Components from the `mdb-react-ui-kit` library for building responsive layouts.
- **Header**: A header component imported from the `components` directory.

ProjectView Component
The

ProjectView

component is a functional component that takes a

state

prop.

Component Definition
```tsx
export default function ProjectView({ state }: { state: any }) {
  return (
    <>
```

```
    {/* Renderização do componente Header com o estado passado como prop */}
    <Header state={state} />
    {/* Secção principal com a classe "list-color" */}
    <section className="list-color">
     {/* Container principal com altura 100% e estilo para overflow e largura da barra de
rolagem */}
      <MDBContainer
       className="h-100"
       style={{ overflow: "auto", scrollbarWidth: "thin" }}
      >
       {/* Linha com classes para centralização e alinhamento dos itens */}
       <MDBRow className="d-flex justify-content-center align-items-center pb-3">
        {/* Coluna com largura para dispositivos grandes (lg) e extra grandes (xl) */}
        <MDBCol lg="8" xl="6"></MDBCol>
       </MDBRow>
      </MDBContainer>
    </section>
   </>
  );
}
```
- **ProjectView**: A functional component that takes a

state

 prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Header Component**: The

Header

 component is rendered with the

state

 prop passed to it.
- **Main Section**: A section element with the class `list-color` for styling.
- **MDBContainer**: A container component from `mdb-react-ui-kit` with a height of 100% and custom styles for overflow and scrollbar width.
- **MDBRow**: A row component from `mdb-react-ui-kit` with classes for centering and aligning items.

- **MDBCol**: A column component from `mdb-react-ui-kit` with responsive width settings for large (

lg

) and extra-large (

xl

) devices.

Summary
The

ProjectView

 component is a simple functional component that renders a header and a main section with a responsive layout. It uses components from the `mdb-react-ui-kit` library for styling and layout, and it passes the

state

 prop to the

Header

 component.

Example Usage
Here is an example of how the

ProjectView

 component might be used in a parent component:
```tsx
import React from 'react';
import ProjectView from './ProjectView';

const App = () => {
  const state = {
    // Define state properties and methods here
  };

  return (
```

```
      <ProjectView state={state} />
    );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state.
- **ProjectView**: The

ProjectView

 component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to manage the project view in the application, ensuring a user-friendly interface with responsive design.

The provided code defines a React class component named `ProfileView` in a TypeScript file, likely named

ProfileView.tsx

. This component allows users to view and edit their profile information, including their name, email, and password. Here's a detailed breakdown of the code:

State Initialization
The component's state includes properties for managing the user's profile information and the view mode (editing or not editing).

handleSubmit Method
The `handleSubmit` method handles the form submission for updating the user's profile information.

State Properties
```tsx
state = {
  name: "", // Nome do utilizador
  email: "", // Email do utilizador
```

```
  password: "", // Palavra-passe do utilizador
  confirmPassword: "", // Confirmação da palavra-passe
  oldName: "", // Nome antigo do utilizador
  oldEmail: "", // Email antigo do utilizador
  view: "not-editing", // Modo de visualização (editando ou não)
  error: null // Erro atual
};
```
- **name**: The user's name.
- **email**: The user's email.
- **password**: The user's password.
- **confirmPassword**: The confirmation of the user's password.
- **oldName**: The user's old name.
- **oldEmail**: The user's old email.
- **view**: The current view mode (either "editing" or "not-editing").
- **error**: The current error state.

handleSubmit Method
```tsx
handleSubmit = () => {
  let user = {
    id: this.props.state.user.id,
    name: this.state.name,
    email: this.state.email,
    password: "null" // Define a palavra-passe como "null" inicialmente
  };

  // Verifica se a palavra-passe e a confirmação da palavra-passe são iguais e não estão vazias
  if (this.state.confirmPassword === this.state.password && this.state.password !== "") {
    user = {
      id: this.props.state.user.id,
      name: this.state.name,
      email: this.state.email,
      password: this.state.password // Atualiza a palavra-passe com o valor fornecido
    };
  } else if (!(this.state.password === "" && this.state.confirmPassword === "")) {
    this.setState({ error: "password" }); // Define o erro como "password" se as palavras-passe não coincidirem
    return;
  }
  // Atualiza as informações do utilizador chamando a função updateUserInfo do estado
  this.props.state.updateUserInfo(user);
  // Define a vista como 'not-editing' e atualiza os valores antigos do nome e email
```

```
    this.setState({
      view: 'not-editing', oldName: this.state.name, oldEmail: this.state.email
    });
};
```
- **user**: An object representing the user, initially with the password set to "null".
- **Password Check**: If the password and confirmPassword match and are not empty, the user object is updated with the provided password. If the passwords do not match, the error state is set to "password" and the function returns early.
- **updateUserInfo**: Calls the

updateUserInfo

 method from the

state

 prop to update the user's information.
- **State Update**: Sets the view to 'not-editing' and updates the oldName and oldEmail with the current name and email.

cancelChanges Method
The

cancelChanges

 method reverts the state to the old values and sets the view to 'not-editing'.
```tsx
cancelChanges = () => {
    this.setState({
      view: 'not-editing', // Define a vista como 'not-editing'
      name: this.state.oldName, // Reverte o nome para o valor antigo
      email: this.state.oldEmail // Reverte o email para o valor antigo
    });
};
```
- **view**: Sets the view to 'not-editing'.
- **name**: Reverts the name to the old value.
- **email**: Reverts the email to the old value.

Export
```tsx
// Exportação do componente ProfileView como o padrão.
export default ProfileView;
```

```
- **default export**: The `ProfileView` component is exported as the default export, making it available for import in other parts of the application.

### Summary
The `ProfileView` component allows users to view and edit their profile information. It includes methods to handle form submission and cancel changes, ensuring that the user's information is updated correctly and that changes can be reverted if needed. The component manages its own state and interacts with the application's state through props.

### Example Usage
Here is an example of how the `ProfileView` component might be used in a parent component:
```tsx
import React from 'react';
import ProfileView from './ProfileView';

const App = () => {
  const state = {
    user: {
      id: 1,
      name: 'John Doe',
      email: 'john.doe@example.com'
    },
    updateUserInfo: (user: { id: number, name: string, email: string, password: string }) => {
      console.log('Updating user info:', user);
    }
  };

  return (
    <ProfileView state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with user information and an

updateUserInfo

 method to manage the application's state.
- **ProfileView**: The `ProfileView` component is rendered with the
```

state

prop passed to it.

This setup provides a clear and structured way to manage user profile information in the application, ensuring a user-friendly interface for editing and updating profile details.

The provided code defines a React class component named

LoginView

in a TypeScript file, likely named

LoginView.tsx

. This component provides a login form for users to enter their email and password. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Form, Button, Alert } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Form, Button, Alert**: Components from the `react-bootstrap` library for building forms, buttons, and alerts.

LoginView Component
The

LoginView

component extends

React.Component

and manages its own state.

State Initialization
```tsx
state = {
```

```
  email: "", // Email do utilizador
  password: "" // Palavra-passe do utilizador
};
```
- **email**: The user's email.
- **password**: The user's password.

handleChange Method
```tsx
handleChange = (e: any) => {
  this.setState({ [e.target.id]: e.target.value });
}
```
- **handleChange**: A method that updates the state when the user types into the input fields. It uses the

id

 attribute of the input elements to determine which state property to update.

render Method
The

render

 method returns the JSX to render the component.

JSX Structure
```tsx
render() {
  return (
    <>
      {/* Título principal com estilo de texto preto e negrito */}
      <h1 className="text-black fw-bolder"> Entrar </h1>
      {/* Div principal com classe para centralização vertical */}
      <div className={"my-auto"}>
        {/* Formulário de login */}
        <Form className={""}>
          {/* Grupo de formulário para o campo Email */}
          <Form.Group className="mb-3 form mx-auto">
            <Form.Label className='text-black'>Endereço de E-mail</Form.Label>
            <Form.Control type="email" id="email" placeholder="examplo@dominio.com"
onChange={this.handleChange}></Form.Control>
```

```
            <Form.Text id="emailHelp">Nunca partilharemos o teu e-mail com
ninguém.</Form.Text>
          </Form.Group>
          {/* Grupo de formulário para o campo Palavra-Passe */}
          <Form.Group className="mb-3 form mx-auto">
            <Form.Label className="text-black">Palavra-Passe</Form.Label>
            <Form.Control type="password" id="password" placeholder="A tua Palavra-
Passe" onChange={this.handleChange} />
          </Form.Group>
          {/* Botão para submeter o formulário de login */}
          <Button className="btn btn-primary" style={{ backgroundColor: "purple",
borderColor: "purple" }} onClick={() => this.props.state.loginUser(this.state.email,
this.state.password)}>Entrar</Button>
        </Form>
        {/* Div para registo com estilo de margem superior */}
        <div className="mt-3">
          <span>Ainda não tem conta? </span>
          <Button className="btn btn-secondary" style={{ backgroundColor: "#FF00FF",
borderColor: "purple", color: "white" }} onClick={() =>
this.props.state.showView("register")}>Registar</Button>
        </div>
        {/* Exibe um alerta se o erro for "userDoesntExist" */}
        {this.props.state.error == "userDoesntExist" && <Alert key={"userDoesntExist"}
variant={"danger"}>Endereço de e-mail não registado.</Alert>}
        {/* Exibe um alerta se o erro for "noMatchPassword" */}
        {this.props.state.error == "noMatchPassword" && <Alert key={"noMatchPassword"}
variant={"danger"}>Palavra-Passe incorreta.</Alert>}
        {/* Botão para a página "Sobre nós" com estilo transparente */}
        <div className="mt-3">
          <Button style={{ backgroundColor: "transparent", borderColor: "transparent",
color: "black" }} className="btn btn-link" onClick={() =>
this.props.state.showView("about")}>Sobre nós</Button>
        </div>
      </div>
    </>
  );
}
```

- **Title**: A heading (

h1

) with the class `text-black fw-bolder` for styling.
- **Main Div**: The main container div with the class `my-auto` for vertical centering.

- **Form**: A form element with input fields for email and password.
- **Email Input**: A form group with a label, input field, and help text for the email.
- **Password Input**: A form group with a label and input field for the password.
- **Login Button**: A button that triggers the

loginUser

 method from the

state

 prop with the email and password from the state.
- **Register Section**: A div with a span and a button that triggers the

showView

 method from the

state

 prop to navigate to the registration view.
- **Error Alerts**: Conditional rendering of alerts if the

error

 state is "userDoesntExist" or "noMatchPassword".
- **About Us Button**: A button that triggers the

showView

 method from the

state

 prop to navigate to the "about" view.

Summary
The

LoginView

 component provides a login form for users to enter their email and password. It includes methods to handle input changes and form submission, ensuring that the user's credentials are passed to the

loginUser

 method from the

state

 prop. The component also provides options for registration and navigating to the "about" page, with error handling for incorrect email or password.

Example Usage
Here is an example of how the

LoginView

 component might be used in a parent component:
```tsx
import React from 'react';
import LoginView from './LoginView';

const App = () => {
  const state = {
    error: null,
    loginUser: (email: string, password: string) => {
      console.log('Logging in with:', email, password);
    },
    showView: (view: string) => {
      console.log(`Navigating to ${view}`);
    }
  };

  return (
    <LoginView state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with

error

,

loginUser

, and

showView

 methods to manage the application's state.
- **LoginView**: The

LoginView

 component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to manage user login in the application, ensuring a user-friendly interface for authentication.

The provided code defines a React functional component named

LoggedMainView

 in a TypeScript file, likely named

LoggedMainView.tsx

. This component serves as the main view for logged-in users, displaying a header, a modal, a list of projects, and a sidebar. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Container, Row, Col } from 'react-bootstrap';
import Modal from '../components/ModalPop';
import Header from '../components/Header';
import ProjectList from '../components/ProjectList';
import Side from '../components/Side';
import { filterListsByCategoryID } from '../state';
```

- **React**: The core React library for building user interfaces.
- **Container, Row, Col**: Components from the `react-bootstrap` library for building responsive layouts.
- **Modal**: A modal component imported from the `components` directory.
- **Header**: A header component imported from the `components` directory.
- **ProjectList**: A component that displays a list of projects, imported from the `components` directory.
- **Side**: A sidebar component imported from the `components` directory.
- **filterListsByCategoryID**: A function imported from the

state

module to filter lists by category ID.

LoggedMainView Component
The

LoggedMainView

component is a functional component that takes a

state

prop.

Component Definition
```tsx
const LoggedMainView = ({ state }: { state: any }) => {
 return (
  <>
   {/* Renderização do Header com o estado passado como prop */}
   <Header state={state} />
   {/* Renderização do Modal com o estado passado como prop */}
   <Modal state={state} />
   {/* Container principal com padding superior */}
   <Container fluid className="p-0" style={{ paddingTop: "70px" }}>
    <Row noGutters>
     {/* Coluna para a lista de projetos */}
     <Col sm={10} className="list-color p-0">
      <ProjectList
       projects={filterListsByCategoryID(
        state.all_lists,
        state.currentCatID
       )}
```

```
          state={state}
         />
        </Col>
        {/* Coluna para a barra lateral */}
        <Col sm={2} className="sidebar-color p-0">
         <Side state={state} />
        </Col>
       </Row>
     </Container>
   </>
 );
};
```
- **LoggedMainView**: A functional component that takes a

state

 prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Header Component**: The

Header

 component is rendered with the

state

 prop passed to it.
- **Modal Component**: The

Modal

 component is rendered with the

state

 prop passed to it.
- **Container**: A container component from `react-bootstrap` with fluid layout and custom padding.
- **Row**: A row component from `react-bootstrap` with no gutters.

- **ProjectList Column**: A column component from `react-bootstrap` that takes up 10/12 of the row's width, with custom classes and padding. It renders the

ProjectList

 component, passing filtered projects and the

state

 prop.
- **Side Column**: A column component from `react-bootstrap` that takes up 2/12 of the row's width, with custom classes and padding. It renders the

Side

 component, passing the

state

 prop.

Export
```tsx
// Exportação do componente LoggedMainView como o padrão.
export default LoggedMainView;
```
- **default export**: The

LoggedMainView

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

LoggedMainView

 component serves as the main view for logged-in users, displaying a header, a modal, a list of projects, and a sidebar. It uses components from `react-bootstrap` for layout and styling, and it passes the

state

prop to child components to manage the application's state.

Example Usage
Here is an example of how the

LoggedMainView

component might be used in a parent component:
```tsx
import React from 'react';
import LoggedMainView from './LoggedMainView';

const App = () => {
  const state = {
    all_lists: [
      // Example list data
    ],
    currentCatID: 1,
    // Other state properties and methods
  };

  return (
    <LoggedMainView state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state, including

all_lists

and

currentCatID

.
- **LoggedMainView**: The

LoggedMainView

component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to manage the main view for logged-in users, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

HelpView

 in a TypeScript file, likely named

HelpView.tsx

. This component renders a modal dialog using Bootstrap components to provide help or instructions to the user. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Modal, Button } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Modal, Button**: Components from the `react-bootstrap` library for building modals and buttons.

HelpView Component
The

HelpView

 component is a functional component that takes

show

 and

handleClose

 as props.

Component Definition
```tsx
const HelpView = ({ show, handleClose }: { show: boolean, handleClose: () => void }) => {
  return (
    // Renderização do Modal do Bootstrap.
    <Modal show={show} onHide={handleClose} centered>
      {/* Cabeçalho do Modal com botão para fechar */}
      <Modal.Header closeButton>
        {/* Título do Modal com estilo de cor roxa */}
        <Modal.Title style={{ color: "purple" }}>Ajuda</Modal.Title>
      </Modal.Header>
      {/* Corpo do Modal */}
      <Modal.Body>
        {/* Parágrafo com instruções sobre a página de disciplinas/projetos */}
        <p style={{ color: "black" }}>
          Na página dedicada às tuas disciplinas/projetos poderás criá-los e adicioná-los às respetivas categorias (Universidade e Pessoal).
          Posteriormente poderás consultá-los e editá-los.
          Após criadas as disciplinas, poderão ser adicionados trabalhos, e dentro dos trabalhos poderão ser adicionadas tarefas.
        </p>
        {/* Parágrafo com instruções sobre a página da Conta */}
        <p style={{ color: "black" }}>
          Na página da tua Conta poderás editar os teus respetivos dados e sair da tua Conta.
        </p>
      </Modal.Body>
      {/* Rodapé do Modal */}
      <Modal.Footer>
        {/* Botão para fechar o Modal com estilo personalizado */}
        <Button
          variant="secondary"
          onClick={handleClose}
          style={{ backgroundColor: "purple", borderColor: "purple", color: "white" }}>
          Fechar
        </Button>
      </Modal.Footer>
    </Modal>
  );
}
```
- **HelpView**: A functional component that takes

show

(a boolean indicating whether the modal is visible) and

handleClose

(a function to close the modal) as props.
- **Modal**: The main container for the modal dialog, with

show

and

onHide

props to control its visibility and handle closing.
- **Modal.Header**: The header section of the modal, with a close button.
- **Modal.Title**: The title of the modal, styled with a purple color.
- **Modal.Body**: The body section of the modal, containing paragraphs with instructions.
- **Modal.Footer**: The footer section of the modal, containing a button to close the modal.
- **Button**: A button to close the modal, styled with custom colors.

Export
```tsx
// Exportação do componente HelpView como o padrão.
export default HelpView;
```
- **default export**: The

HelpView

component is exported as the default export, making it available for import in other parts of the application.

Summary
The

HelpView

component provides a modal dialog with help or instructions for the user. It uses Bootstrap components for layout and styling, and it takes

show

and

handleClose

as props to control the modal's visibility and handle closing.

Example Usage
Here is an example of how the

HelpView

component might be used in a parent component:
```tsx
import React, { useState } from 'react';
import HelpView from './HelpView';
import Button from 'react-bootstrap/Button';

const App = () => {
  const [showHelp, setShowHelp] = useState(false);

  const handleShowHelp = () => setShowHelp(true);
  const handleCloseHelp = () => setShowHelp(false);

  return (
    <div>
      <Button onClick={handleShowHelp}>Show Help</Button>
      <HelpView show={showHelp} handleClose={handleCloseHelp} />
    </div>
  );
};

export default App;
```
In this example:
- **showHelp**: A state variable to control the visibility of the

HelpView

modal.
- **handleShowHelp**: A function to show the

HelpView

 modal.
- **handleCloseHelp**: A function to close the

HelpView

 modal.
- **HelpView**: The

HelpView

 component is rendered with the

show

 and

handleClose

 props passed to it.

This setup provides a clear and structured way to display help or instructions to the user in a modal dialog, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

AboutView

 in a TypeScript file, likely named

AboutView.tsx

. This component displays information about the team and the project, and includes a button to navigate back to the welcome view. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import Button from 'react-bootstrap/Button';
```
- **React**: The core React library for building user interfaces.

- **Button**: A button component from the `react-bootstrap` library for building styled buttons.

AboutView Component
The

AboutView

component is a functional component that takes a

state

prop.

Component Definition
```tsx
const AboutView = ({ state }: { state: any }) => {
  return (
    // Div principal com a classe "about-container".
    <div className="about-container">
      {/* Botão para voltar à vista de boas-vindas */}
      <Button
        style={{ position: 'absolute', top: '10px', left: '10px', backgroundColor: "purple", borderColor: "purple" }}
        onClick={() => state.showView("welcome")}
      >
        Voltar
      </Button>
      {/* Título principal */}
      <h1>Sobre Nós</h1>
      {/* Parágrafo com informações sobre a equipa e o projeto */}
      <p className="about-text">
        Somos três colegas apaixonados por tecnologia!<br />
        Este website foi criado no âmbito da cadeira de Desenvolvimento de Aplicações para a Web e reflete o nosso empenho em aplicar<br /> o que aprendemos em sala de aula para resolver desafios reais.<br />
        Sabemos como pode ser difícil equilibrar os estudos, atividades extracurriculares e a vida pessoal.<br /> Por isso, decidimos criar esta plataforma dedicada a ajudar estudantes a organizarem a sua vida académica de forma prática e eficiente.<br />
        O nosso objetivo é oferecer ferramentas simples e úteis que ajudem outros estudantes a gerir melhor o seu tempo.<br />
        <br />
        <span>Trabalho realizado por: <br />
          Helena Vassal Nº:75525 <br />
```

```
            Rui Saraiva Nº:79890 <br />
            Vasco Evaristo Nº:79806</span>
        </p>
      </div>
    );
}
```
- **AboutView**: A functional component that takes a

state

 prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Main Div**: The main container div with the class `about-container` for styling.
- **Back Button**: A button with custom styles and an

onClick

 handler that triggers the

showView

 method from the

state

 prop to navigate back to the welcome view.
- **Title**: A heading (

h1

) with the text "Sobre Nós".
- **Description Paragraph**: A paragraph with the class `about-text` containing information about the team and the project. It includes line breaks (`<br />`) for formatting and a span with the names and numbers of the team members.

Export
```tsx
// Exportação do componente AboutView como o padrão.
export default AboutView;
```

- **default export**: The

AboutView

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

AboutView

 component provides information about the team and the project, and includes a button to navigate back to the welcome view. It uses Bootstrap components for styling and layout, and it takes a

state

 prop to manage navigation.

Example Usage
Here is an example of how the

AboutView

 component might be used in a parent component:
```tsx
import React from 'react';
import AboutView from './AboutView';

const App = () => {
  const state = {
    showView: (view: string) => {
      console.log(`Navigating to ${view}`);
    }
  };

  return (
    <AboutView state={state} />
  );
};

export default App;
```

In this example:
- **state**: An object with a

showView

 method to manage the application's state.
- **AboutView**: The

AboutView

 component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to display information about the team and the project, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

Task

 in a TypeScript file, likely named

Task.tsx

. This component renders a task card using MDB (Material Design for Bootstrap) components, displaying the task's title, description, and state, along with actions to edit, delete, or complete the task. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { MDBCard, MDBCardBody, MDBCardFooter, MDBBadge, MDBBtn } from 'mdb-react-ui-kit';
import { ITask } from './interfaces'; // Assuming ITask is defined in interfaces.ts
```
- **React**: The core React library for building user interfaces.
- **MDBCard, MDBCardBody, MDBCardFooter, MDBBadge, MDBBtn**: Components from the `mdb-react-ui-kit` library for building styled cards, badges, and buttons.

- **ITask**: An interface representing the structure of a task object, imported from a local file.

Task Component
The

Task

component is a functional component that takes

task

and

state

as props.

Component Definition
```tsx
const Task = ({ task, state }: { task: ITask, state: any }) => {
  return (
    <>
      {/* Cartão do MDB com margem inferior e largura total */}
      <MDBCard className={"mb-4 w-100"} style={{ background: "#cc55ff" }}>
        <MDBCardBody>
          {/* Div para alinhar os itens horizontalmente e justificar o espaço entre eles */}
          <div className='d-flex justify-content-between align-items-center'>
            {/* Div para alinhar os itens horizontalmente */}
            <div className='d-flex align-items-center' style={{ overflowBlock: "hidden" }}>
              <div className='ms-3'>
                {/* Título da tarefa */}
                <p className='fw-normal text-start'>{task.title}</p>
                {/* Descrição da tarefa, se existir */}
                {task.description !== "" &&
                  <>
                    <hr className={"divider"} />
                    <p className='text-white small mb-0 text-start'>{task.description}</p>
                  </>
                }
              </div>
            </div>
            {/* Badge para o estado da tarefa */}
            {task.state === "Acabada" &&
```

```
                <MDBBadge pill color='success' light>
                  {task.state}
                </MDBBadge>
              }
              {task.state === "Em Curso" &&
                <MDBBadge pill color='warning' light>
                  {task.state}
                </MDBBadge>
              }
              {task.state === "Por Começar" &&
                <MDBBadge pill color='danger' light>
                  {task.state}
                </MDBBadge>
              }
            </div>
          </MDBCardBody>
          {/* Rodapé do cartão com botões e dropdown */}
          <MDBCardFooter border='0' className='p-2 d-flex justify-content-around'>
            <div className="dropdown">
              {/* Botão para abrir o dropdown */}
              <button className="btn btn-secondary dropdown-toggle"
                type="button"
                id="categoryType" data-bs-toggle="dropdown"
              >
                Ações
              </button>
              {/* Menu dropdown com opções */}
              <ul className="dropdown-menu" aria-labelledby="categoryType">
                <li><a className="dropdown-item" href="#">Editar</a></li>
                <li><a className="dropdown-item" href="#">Apagar</a></li>
              </ul>
            </div>
            {/* Botão para marcar a tarefa como concluída */}
            <MDBBtn color="success" onClick={() =>
state.completeTask(task.id)}>Concluir</MDBBtn>
          </MDBCardFooter>
        </MDBCard>
      </>
    );
}
```

- **Task**: A functional component that takes

task

(an object representing the task) and

state

(an object containing methods and properties to manage the application's state) as props.
- **MDBCard**: The main container for the task card, with custom styles for margin and background color.
- **MDBCardBody**: The body section of the card, containing the task's title and description.
- **MDBCardFooter**: The footer section of the card, containing action buttons and a dropdown menu.
- **MDBBadge**: A badge component to display the task's state with different colors based on the state.
- **MDBBtn**: A button to mark the task as completed.

JSX Structure
- **Main Card**: The main container card with custom styles.
- **Card Body**: The body section containing:
  - **Title**: The task's title.
  - **Description**: The task's description, if it exists.
  - **State Badge**: A badge indicating the task's state (Acabada, Em Curso, Por Começar).
- **Card Footer**: The footer section containing:
  - **Dropdown Menu**: A dropdown menu with options to edit or delete the task.
  - **Complete Button**: A button to mark the task as completed, triggering the

completeTask

method from the

state

prop.

Export
Ensure the component is exported correctly:
```tsx
export default Task;
```
- **default export**: The

Task

component is exported as the default export, making it available for import in other parts of the application.

Summary
The

Task

component provides a styled card to display a task's title, description, and state, along with actions to edit, delete, or complete the task. It uses MDB components for layout and styling, and it takes

task

and

state

as props to manage the task's data and actions.

Example Usage
Here is an example of how the

Task

component might be used in a parent component:
```tsx
import React from 'react';
import Task from './Task';
import { ITask } from './interfaces';

const App = () => {
  const state = {
    completeTask: (id: number) => {
      console.log(`Completing task with id: ${id}`);
    }
  };

  const task: ITask = {
    id: 1,
    title: 'Example Task',
    description: 'This is an example task description.',
    state: 'Em Curso'
  };
```

```
  return (
    <Task task={task} state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with a

completeTask

 method to manage the application's state.
- **task**: An object representing a task with properties

id

,

title

,

description

, and

state

.
- **Task**: The

Task

 component is rendered with the

task

 and

state

 props passed to it.

This setup provides a clear and structured way to display and manage tasks in the application, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

Side

in a TypeScript file, likely named

SideBar.tsx

. This component renders a sidebar with a list of categories and a button to add a new category or project. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Button } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Button**: A button component from the `react-bootstrap` library for building styled buttons.

Side Component
The

Side

component is a functional component that takes

state

as a prop.

Component Definition
```tsx
const Side = ({ state }: { state: any }) => {
  return (
    <div className="sidebar">
```

```
    <ul className="list-unstyled">
     <li>
      {state.types.map((type: string) => {
       return (
        <>
         <h5 className="text-white">{type}</h5>
         <ul className="list-unstyled">
          {getArray(state.categories, type).map((category: any) => {
           return (
            <li key={category.id}>
             <a
              href="#"
              className="text-white"
              onClick={() => state.updateCat(category.id)}
             >
              {category.title}
             </a>
            </li>
           );
          })}
          {getArray(state.categories, type).length === 0 && (
           <p className="text-white small"> Vazio </p>
          )}
         </ul>
         <div className="my-2"></div>
        </>
       );
      })}
     </li>
    </ul>
    <Button
     className="btn btn-block btn-sm btn-light btn-noglow"
     style={{ width: "100%" }}
     onClick={() => state.showModalView("createCategory")}
    >
     Adicionar Disciplina/Projeto
    </Button>
   </div>
  );
};
```

- **Side**: A functional component that takes

state

as a prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Main Div**: The main container div with the class `sidebar` for styling.
- **Unordered List**: An unordered list with the class `list-unstyled` for styling.
- **List Item**: A list item that maps over

state.types

to render each type.
 - **Type Header**: A header (`h5`) with the class `text-white` for each type.
 - **Nested Unordered List**: A nested unordered list with the class `list-unstyled` for each type's categories.
   - **Category List Item**: A list item for each category, with a link to update the category.
   - **Empty Message**: A paragraph with the class `text-white small` that displays "Vazio" if there are no categories for the type.
 - **Divider**: A div with the class `my-2` for spacing between types.
- **Add Button**: A button with custom styles and classes to add a new category or project, triggering the

showModalView

method from the

state

prop.

Helper Function
Ensure you have the

getArray

helper function defined somewhere in your code:
```tsx
const getArray = (categories: any[], type: string) => {
  return categories.filter(category => category.type === type);
};
```
- **getArray**: A function that filters the categories based on the type.

Export
Ensure the component is exported correctly:
```tsx
export default Side;
```
- **default export**: The

Side

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

Side

 component provides a sidebar with a list of categories and a button to add a new category or project. It uses Bootstrap components for layout and styling, and it takes

state

 as a prop to manage the application's state.

Example Usage
Here is an example of how the

Side

 component might be used in a parent component:
```tsx
import React from 'react';
import Side from './SideBar';

const App = () => {
  const state = {
    types: ['Universidade', 'Pessoal'],
    categories: [
      { id: 1, title: 'Matemática', type: 'Universidade' },
      { id: 2, title: 'Física', type: 'Universidade' },
      { id: 3, title: 'Ginástica', type: 'Pessoal' }
    ],
    updateCat: (id: number) => {
      console.log(`Updating category with id: ${id}`);
```

```
    },
    showModalView: (view: string) => {
      console.log(`Showing modal view: ${view}`);
    }
  };

  return (
    <Side state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state, including `types`,

categories

, `updateCat`, and

showModalView

.
- **Side**: The

Side

component is rendered with the

state

prop passed to it.

This setup provides a clear and structured way to manage and display categories in a sidebar, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

ProjectList

in a TypeScript file, likely named

ProjectList.tsx

. This component renders a list of projects using MDB (Material Design for Bootstrap) components. It displays a message when there are no projects and iterates over the list of projects to render individual project cards. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { MDBContainer, MDBRow, MDBCol } from 'mdb-react-ui-kit';
import ProjectCard from './ProjectCard'; // Assuming ProjectCard is defined in ProjectCard.tsx
import { ProjectListProps } from './interfaces'; // Assuming ProjectListProps is defined in interfaces.ts
```
- **React**: The core React library for building user interfaces.
- **MDBContainer, MDBRow, MDBCol**: Components from the `mdb-react-ui-kit` library for building responsive layouts.
- **ProjectCard**: A component that renders individual project cards, imported from a local file.
- **ProjectListProps**: An interface representing the props for the

ProjectList

component, imported from a local file.

ProjectList Component
The

ProjectList

component is a functional component that takes

projects

and

state

as props.

Component Definition
```tsx
export default function ProjectList({ projects, state }: ProjectListProps) {
  return (
    // Secção principal com a classe "list-color".
    <section className="list-color">
      {/* Container principal com altura 100% e estilo para overflow e largura da barra de rolagem */}
      <MDBContainer
        className="h-100"
        style={{ overflow: "auto", scrollbarWidth: "thin" }}
      >
        {/* Linha com classes para centralização e alinhamento dos itens */}
        <MDBRow className="d-flex justify-content-center align-items-center pb-3">
          <MDBCol lg="8" xl="6">
            {/* Mensagem quando não existem projetos */}
            {projects.length === 0 && (
              <div className="text-center" style={{ color: "white" }}>
                <h2>Não existem novos trabalhos.</h2>
              </div>
            )}
            {/* Iteração sobre a lista de projetos e renderização do componente ProjectCard */}
            {projects.map((project) => (
              <ProjectCard key={project.id} project={project} state={state} />
            ))}
          </MDBCol>
        </MDBRow>
      </MDBContainer>
    </section>
  );
}
```
- **ProjectList**: A functional component that takes

projects

 (an array of project objects) and

state

 (an object containing methods and properties to manage the application's state) as props.
- **MDBContainer**: The main container for the project list, with custom styles for height, overflow, and scrollbar width.

- **MDBRow**: A row component from `mdb-react-ui-kit` with classes for centering and aligning items.
- **MDBCol**: A column component from `mdb-react-ui-kit` with responsive width settings for large (

lg

) and extra-large (

xl

) devices.
- **ProjectCard**: A component that renders individual project cards, iterated over the

projects

 array.

JSX Structure
- **Main Section**: The main section with the class `list-color` for styling.
- **Container**: A container component from `mdb-react-ui-kit` with custom styles.
- **Row**: A row component from `mdb-react-ui-kit` with classes for centering and aligning items.
- **Column**: A column component from `mdb-react-ui-kit` with responsive width settings.
- **No Projects Message**: A div with a message displayed when there are no projects.
- **Project Cards**: Iteration over the

projects

 array to render individual

ProjectCard

 components, passing

project

 and

state

 as props.

Props Interface
Ensure you have the

ProjectListProps

 interface defined somewhere in your code:
```tsx
export interface ProjectListProps {
  projects: Array<{ id: number; title: string; description: string; state: string }>;
  state: any;
}
```
- **ProjectListProps**: An interface representing the props for the

ProjectList

 component, including

projects

 (an array of project objects) and

state

 (an object containing methods and properties to manage the application's state).

Export
Ensure the component is exported correctly:
```tsx
export default ProjectList;
```
- **default export**: The

ProjectList

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

ProjectList

component provides a styled list of projects using MDB components. It displays a message when there are no projects and iterates over the list of projects to render individual project cards. It takes

projects

and

state

as props to manage the project's data and actions.

Example Usage
Here is an example of how the

ProjectList

component might be used in a parent component:
```tsx
import React from 'react';
import ProjectList from './ProjectList';
import { ProjectListProps } from './interfaces';

const App = () => {
  const state = {
    // Define state properties and methods here
  };

  const projects: ProjectListProps['projects'] = [
    { id: 1, title: 'Project 1', description: 'Description 1', state: 'Active' },
    { id: 2, title: 'Project 2', description: 'Description 2', state: 'Completed' }
  ];

  return (
    <ProjectList projects={projects} state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state.
- **projects**: An array of project objects with properties `id`, `title`, `description`, and

state

.
- **ProjectList**: The

ProjectList

component is rendered with the

projects

and

state

props passed to it.

This setup provides a clear and structured way to display and manage a list of projects in the application, ensuring a user-friendly interface with responsive design and state management.

The provided code snippet indicates that the file

ModalPop.tsx

contains a component named `ModalPop`. This component is likely used to display modal dialogs in the application. Here's a detailed breakdown of how such a component might be structured and used:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { Modal, Button } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Modal, Button**: Components from the `react-bootstrap` library for building modals and buttons.

ModalPop Component
The `ModalPop` component is a functional component that takes `show`, `handleClose`, and `content` as props.

Component Definition
```tsx
const ModalPop = ({ show, handleClose, content }: { show: boolean, handleClose: () =>
void, content: React.ReactNode }) => {
  return (
    // Renderização do Modal do Bootstrap.
    <Modal show={show} onHide={handleClose} centered>
      {/* Cabeçalho do Modal com botão para fechar */}
      <Modal.Header closeButton>
        {/* Título do Modal */}
        <Modal.Title>Modal Title</Modal.Title>
      </Modal.Header>
      {/* Corpo do Modal */}
      <Modal.Body>
        {content}
      </Modal.Body>
      {/* Rodapé do Modal */}
      <Modal.Footer>
        {/* Botão para fechar o Modal */}
        <Button variant="secondary" onClick={handleClose}>
          Fechar
        </Button>
      </Modal.Footer>
    </Modal>
  );
}
```
- **ModalPop**: A functional component that takes `show` (a boolean indicating whether the modal is visible), `handleClose` (a function to close the modal), and `content` (the content to display inside the modal) as props.
- **Modal**: The main container for the modal dialog, with `show` and `onHide` props to control its visibility and handle closing.
- **Modal.Header**: The header section of the modal, with a close button.
- **Modal.Title**: The title of the modal.
- **Modal.Body**: The body section of the modal, containing the `content` prop.
- **Modal.Footer**: The footer section of the modal, containing a button to close the modal.
- **Button**: A button to close the modal.

Export
Ensure the component is exported correctly:
```tsx
export default ModalPop;
```

```
- **default export**: The `ModalPop` component is exported as the default export, making it available for import in other parts of the application.

### Summary
The `ModalPop` component provides a modal dialog using Bootstrap components. It takes `show`, `handleClose`, and `content` as props to control the modal's visibility, handle closing, and display content inside the modal.

### Example Usage
Here is an example of how the `ModalPop` component might be used in a parent component:
```tsx
import React, { useState } from 'react';
import ModalPop from './ModalPop';
import Button from 'react-bootstrap/Button';

const App = () => {
  const [showModal, setShowModal] = useState(false);

  const handleShowModal = () => setShowModal(true);
  const handleCloseModal = () => setShowModal(false);

  return (
    <div>
      <Button onClick={handleShowModal}>Show Modal</Button>
      <ModalPop show={showModal} handleClose={handleCloseModal} content={<p>This is the modal content.</p>} />
    </div>
  );
};

export default App;
```
In this example:
- **showModal**: A state variable to control the visibility of the `ModalPop` component.
- **handleShowModal**: A function to show the `ModalPop` component.
- **handleCloseModal**: A function to close the `ModalPop` component.
- **ModalPop**: The `ModalPop` component is rendered with the `show`, `handleClose`, and `content` props passed to it.

This setup provides a clear and structured way to display modal dialogs in the application, ensuring a user-friendly interface with responsive design and state management.
```

The provided code defines a React functional component named

List

in a TypeScript file, likely named

List.tsx

. This component renders a list of items using MDB (Material Design for Bootstrap) components. It includes functionality to delete a list and navigate to a different view. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { MDBContainer, MDBRow, MDBCol, MDBCard, MDBCardBody, MDBListGroup, MDBBtn } from 'mdb-react-ui-kit';
```
- **React**: The core React library for building user interfaces.
- **MDBContainer, MDBRow, MDBCol, MDBCard, MDBCardBody, MDBListGroup, MDBBtn**: Components from the `mdb-react-ui-kit` library for building responsive layouts and styled elements.

List Component
The

List

component is a functional component that takes

state

as a prop.

Component Definition
```tsx
const List = ({ state }: { state: any }) => {
  return (
    <>
      {/* Secção principal com a classe "list-color" */}
      <section className="list-color">
```

```
      {/* Container principal com altura 100% e estilo para overflow e largura da barra de rolagem */}
      <MDBContainer className="h-100" style={{ overflow: "auto", scrollbarWidth: "thin" }}>
       {/* Linha com classes para centralização e alinhamento dos itens */}
       <MDBRow className="d-flex justify-content-center align-items-center pb-3">
        <MDBCol lg="8" xl="6">
         <MDBCard>
          <MDBCardBody>
           {/* Lista de itens */}
           {state.currentList && (
            <MDBListGroup>
             {/* Botão para eliminar a lista */}
             <MDBBtn
              className="btn-danger btn-noglow"
              onClick={() => {
               state.deleteList(state.currentList.id);
               state.showView("logged");
              }}
             >
              Eliminar Trabalho
             </MDBBtn>
            </MDBListGroup>
           )}
          </MDBCardBody>
         </MDBCard>
        </MDBCol>
       </MDBRow>
      </MDBContainer>
    </section>
   </>
  );
};
```

- **List**: A functional component that takes

state

 as a prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.

JSX Structure
- **Main Section**: The main section with the class `list-color` for styling.

- **Container**: A container component from `mdb-react-ui-kit` with custom styles for height, overflow, and scrollbar width.
- **Row**: A row component from `mdb-react-ui-kit` with classes for centering and aligning items.
- **Column**: A column component from `mdb-react-ui-kit` with responsive width settings.
- **Card**: A card component from `mdb-react-ui-kit` to contain the list items.
- **Card Body**: The body section of the card.
- **List Group**: A list group component from `mdb-react-ui-kit` to contain the list items.
- **Delete Button**: A button to delete the current list, triggering the

deleteList

 method from the

state

 prop and navigating to the "logged" view.

Export
Ensure the component is exported correctly:
```tsx
export default List;
```
- **default export**: The

List

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

List

 component provides a styled list of items using MDB components. It includes functionality to delete a list and navigate to a different view. It takes

state

 as a prop to manage the list's data and actions.

Example Usage

Here is an example of how the

List

 component might be used in a parent component:
```tsx
import React from 'react';
import List from './List';

const App = () => {
  const state = {
    currentList: { id: 1, title: 'Example List' },
    deleteList: (id: number) => {
      console.log(`Deleting list with id: ${id}`);
    },
    showView: (view: string) => {
      console.log(`Navigating to view: ${view}`);
    }
  };

  return (
    <List state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state, including

currentList

,

deleteList

, and

showView

.
- **List**: The

List

component is rendered with the

state

prop passed to it.

This setup provides a clear and structured way to display and manage a list of items in the application, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

Header

in a TypeScript file, likely named

Header.tsx

. This component renders a header with a navigation bar, a brand logo, and a help button that triggers a modal dialog for help. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React, { useState } from 'react';
import { Navbar, Nav, Container } from 'react-bootstrap';
import HelpView from './HelpView'; // Assuming HelpView is defined in HelpView.tsx
import logo from '../assets/logo.png'; // Assuming logo.png is in the assets folder
import help_button from '../assets/help_button.png'; // Assuming help_button.png is in the assets folder
```
- **React**: The core React library for building user interfaces.
- **useState**: A React hook for managing state in functional components.
- **Navbar, Nav, Container**: Components from the `react-bootstrap` library for building responsive navigation bars.
- **HelpView**: A component that renders a help modal, imported from a local file.
- **logo**: The application's logo image, imported from the assets folder.
- **help_button**: The help button image, imported from the assets folder.

Header Component

The

Header

component is a functional component that takes

state

as a prop.

Component Definition
```tsx
const Header = ({ state }: { state: any }) => {
  const [showHelp, setShowHelp] = useState(false);

  // Função para mostrar o HelpView.
  const handleShowHelp = () => setShowHelp(true);
  // Função para fechar o HelpView.
  const handleCloseHelp = () => setShowHelp(false);

  return (
    // Cabeçalho principal com a classe "header-top".
    <header className={"header-top"}>
      <div>
        {/* Navbar do Bootstrap com variante branca e expansão em lg */}
        <Navbar variant={"white"} expand="lg" className="p-0">
          <Container className="p-0">
            {/* Marca da Navbar com clique para mudar de vista */}
            <Navbar.Brand
              style={{ color: "white" }}
              onClick={() => state.isLoggedIn ? state.showView("logged") :
state.showView("welcome")}
            >
              {/* Logotipo da aplicação */}
              <img src={logo} height={"55"} alt="Logo" />UAlly
            </Navbar.Brand>
            {/* Navegação à direita */}
            <Nav className={"ml-auto"}>
              {/* Link para mostrar o HelpView */}
              <Nav.Link onClick={handleShowHelp}>
                <img src={help_button} alt="Help" height="30" />
              </Nav.Link>
            </Nav>
          </Container>
```

```
            </Navbar>
          </div>
          {/* Componente HelpView para mostrar a ajuda */}
          <HelpView show={showHelp} handleClose={handleCloseHelp} />
        </header>
    );
}
```
- **Header**: A functional component that takes

state

 as a prop.
- **state**: An object passed as a prop, likely containing methods and properties to manage the application's state.
- **showHelp**: A state variable to control the visibility of the

HelpView

 modal.
- **handleShowHelp**: A function to show the

HelpView

 modal.
- **handleCloseHelp**: A function to close the

HelpView

 modal.

JSX Structure
- **Main Header**: The main header with the class

header-top

 for styling.
- **Navbar**: A navigation bar component from `react-bootstrap` with custom styles and properties.
  - **Container**: A container component from `react-bootstrap` with custom styles.
  - **Navbar.Brand**: The brand section of the navbar, with an

onClick

handler to navigate based on the

isLoggedIn

state.
  - **Logo**: An image element displaying the application's logo.
- **Nav**: A navigation section aligned to the right.
  - **Nav.Link**: A link element to show the

HelpView

modal, displaying the help button image.
- **HelpView**: The

HelpView

component rendered with the

show

and

handleClose

props.

Export
Ensure the component is exported correctly:
```tsx
export default Header;
```
- **default export**: The

Header

component is exported as the default export, making it available for import in other parts of the application.

Summary
The

Header

component provides a styled header with a navigation bar, a brand logo, and a help button that triggers a modal dialog for help. It uses Bootstrap components for layout and styling, and it takes

state

as a prop to manage navigation and modal visibility.

Example Usage
Here is an example of how the

Header

component might be used in a parent component:
```tsx
import React from 'react';
import Header from './Header';

const App = () => {
  const state = {
    isLoggedIn: true,
    showView: (view: string) => {
      console.log(`Navigating to ${view}`);
    }
  };

  return (
    <Header state={state} />
  );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state, including

isLoggedIn

and

showView

.

- **Header**: The

Header

 component is rendered with the

state

 prop passed to it.

This setup provides a clear and structured way to display a header with navigation and help functionality in the application, ensuring a user-friendly interface with responsive design and state management.

The provided code defines a React functional component named

Category

 in a TypeScript file, likely named

Category.tsx

. This component renders a category item with a folder icon, a title, and a close button to delete the category. Here's a detailed breakdown of the code:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import { CloseButton } from 'react-bootstrap';
import { ICategory } from '../Categories';
import folderIcon from '../assets/images/folder-symbol.png';
```
- **React**: The core React library for building user interfaces.
- **CloseButton**: A close button component from the `react-bootstrap` library.
- **ICategory**: An interface representing the structure of a category object, imported from a local file.
- **folderIcon**: The folder icon image, imported from the assets folder.

Category Component
The

Category

component is a functional component that takes

category

and

state

as props.

Component Definition
```tsx
export default function Category({
  category,
  state,
}: {
  category: ICategory;
  state: any;
}) {
  return (
    <a
      href={`#${category.id}`}
      data-toggle="collapse"
      aria-expanded="false"
      onClick={() => state.updateCat(category.id)}
    >
      {/* Ícone de pasta */}
      <img
        src={folderIcon}
        style={{
          width: "20px",
          marginRight: "10px",
        }}
        alt="Folder Icon"
      />
      {category.title}
      {/* Botão para apagar a categoria */}
      <CloseButton
        variant={"#FFF6F6"}
        style={{ float: "right" }}
        onClick={() => state.deleteCategory(category.id)}
      />
```

```
    </a>
  );
}
```
- **Category**: A functional component that takes

category

 (an object representing the category) and

state

 (an object containing methods and properties to manage the application's state) as props.
- **a**: An anchor element that acts as a collapsible link, triggering the

updateCat

 method from the

state

 prop when clicked.
- **img**: An image element displaying the folder icon, with custom styles for width and margin.
- **CloseButton**: A close button to delete the category, triggering the

deleteCategory

 method from the

state

 prop when clicked.

JSX Structure
- **Anchor Element**: The main container for the category item, with attributes for collapsible behavior and an

onClick

 handler to update the category.
- **Folder Icon**: An image element displaying the folder icon, with custom styles.
- **Category Title**: The title of the category, displayed as text.
- **Close Button**: A close button to delete the category, with custom styles and an

onClick

 handler.

Props Interface
Ensure you have the

ICategory

 interface defined somewhere in your code:
```tsx
export interface ICategory {
  id: number;
  title: string;
  description: string;
  userID: number;
  type: string;
}
```
- **ICategory**: An interface representing the structure of a category object, including

id

,

title

, `description`, `userID`, and `type`.

Export
Ensure the component is exported correctly:
```tsx
export default Category;
```
- **default export**: The

Category

 component is exported as the default export, making it available for import in other parts of the application.

Summary
The

Category

component provides a styled category item with a folder icon, a title, and a close button to delete the category. It uses Bootstrap components for layout and styling, and it takes

category

and

state

as props to manage the category's data and actions.

Example Usage
Here is an example of how the

Category

component might be used in a parent component:
```tsx
import React from 'react';
import Category from './Category';
import { ICategory } from '../Categories';

const App = () => {
  const state = {
    updateCat: (id: number) => {
      console.log(`Updating category with id: ${id}`);
    },
    deleteCategory: (id: number) => {
      console.log(`Deleting category with id: ${id}`);
    }
  };

  const category: ICategory = {
    id: 1,
    title: 'Example Category',
    description: 'This is an example category.',
    userID: 1,
    type: 'Example Type'
  };

  return (
```

```
      <Category category={category} state={state} />
    );
};

export default App;
```
In this example:
- **state**: An object with properties and methods to manage the application's state, including

updateCat

 and

deleteCategory

.
- **category**: An object representing a category with properties

id

,

title

, `description`, `userID`, and `type`.
- **Category**: The

Category

 component is rendered with the

category

 and

state

 props passed to it.

This setup provides a clear and structured way to display and manage categories in the application, ensuring a user-friendly interface with responsive design and state management.

The provided code snippet indicates that the file

BaseLayout.tsx

 contains a component named `BaseLayout`. This component is likely the main layout component for the application, providing a structure that includes a header, main content area, and possibly other common elements. Here's a detailed breakdown of how such a component might be structured and used:

Imports
Ensure you have the necessary imports at the top of your file:
```tsx
import React from 'react';
import Header from './Header'; // Assuming Header is defined in Header.tsx
import { Container } from 'react-bootstrap';
```
- **React**: The core React library for building user interfaces.
- **Header**: A header component imported from a local file.
- **Container**: A container component from the `react-bootstrap` library for building responsive layouts.

BaseLayout Component
The `BaseLayout` component is a functional component that takes `children` as props.

Component Definition
```tsx
const BaseLayout = ({ children }: { children: React.ReactNode }) => {
  return (
    <>
      {/* Renderização do Header */}
      <Header />
      {/* Container principal para o conteúdo */}
      <Container fluid className="main-content">
        {children}
      </Container>
    </>
  );
};
```
- **BaseLayout**: A functional component that takes `children` (the content to be rendered inside the layout) as props.

The provided CSS code defines global styles for the `body` element of your web application. Here's a detailed breakdown of the code:

CSS Code
```css
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen',
    'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue',
    sans-serif;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}
```

Explanation

`margin: 0;`
- **Purpose**: Removes the default margin from the `body` element, ensuring that the content starts from the very edge of the viewport.

`font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Oxygen', 'Ubuntu', 'Cantarell', 'Fira Sans', 'Droid Sans', 'Helvetica Neue', sans-serif;`
- **Purpose**: Sets the font family for the entire document. This is a list of font families that the browser will try to use in the specified order.
  - **-apple-system**: The system font for macOS and iOS.
  - **BlinkMacSystemFont**: The system font for macOS in Blink-based browsers (like Chrome).
  - **'Segoe UI'**: The system font for Windows.
  - **'Roboto'**: A popular sans-serif font used in many Android and web applications.
  - **'Oxygen'**, **'Ubuntu'**, **'Cantarell'**, **'Fira Sans'**, **'Droid Sans'**, **'Helvetica Neue'**: Other popular sans-serif fonts.
  - **sans-serif**: A generic fallback font family.

`-webkit-font-smoothing: antialiased;`
- **Purpose**: Improves the appearance of text by smoothing the edges of the characters on WebKit-based browsers (like Safari and Chrome).

`-moz-osx-font-smoothing: grayscale;`
- **Purpose**: Improves the appearance of text by smoothing the edges of the characters on macOS in Firefox.

Summary

This CSS code sets global styles for the `body` element of your web application. It removes the default margin, sets a list of preferred font families, and applies font smoothing for better text rendering. These styles ensure a consistent and visually appealing appearance across different browsers and operating systems.

Example Usage
Here is an example of how this CSS might be used in an HTML file:
```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Web Application</title>
  <link rel="stylesheet" href="index.css">
</head>
<body>
  <h1>Welcome to My Web Application</h1>
  <p>This is a sample paragraph to demonstrate the global styles applied to the body element.</p>
</body>
</html>
```
In this example:
- **index.css**: The CSS file containing the provided styles is linked in the HTML file.
- **body**: The global styles are applied to the `body` element, affecting all content within the body.

This setup ensures a consistent and visually appealing appearance for your web application, with smooth text rendering and a clean layout.

Similar code found with 2 license types

The provided CSS code defines a class named `.todoList` that centers the text within elements that have this class applied. Here's a detailed breakdown of the code:

CSS Code
```css
.todoList {
   text-align: center;
}
```

Explanation

`.todoList`
- **Selector**: The `.todoList` selector targets all HTML elements with the class `todoList`.

`text-align: center;`
- **Property**: The `text-align` property specifies the horizontal alignment of text within the element.
- **Value**: The value `center` centers the text horizontally within the element.

Summary
This CSS code centers the text within any HTML element that has the class `todoList` applied. This is useful for ensuring that the text content of elements such as lists, divs, or other containers is centered horizontally.

Example Usage
Here is an example of how this CSS might be used in an HTML file:
```html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Todo List</title>
 <link rel="stylesheet" href="BaseLayout.css">
</head>
<body>
 <div class="todoList">
  <h1>My Todo List</h1>
  <ul>
   <li>Buy groceries</li>
   <li>Complete homework</li>
   <li>Call the bank</li>
  </ul>
 </div>
</body>
</html>
```
In this example:
- **BaseLayout.css**: The CSS file containing the provided styles is linked in the HTML file.
- **todoList**: The `todoList` class is applied to a `div` element, centering the text within the `div`, including the heading and list items.

This setup ensures that the text content within elements with the `todoList` class is centered horizontally, providing a clean and visually appealing layout.

Similar code found with 2 license types

The provided CSS code defines various classes for styling elements in your application. Here's a detailed breakdown of each class and its purpose:

CSS Code
```css
.welcomeview {
  height: 80vh;
}

.about-text {
  color: rgb(0, 0, 0);
}

.divider {
  background-color: #FFF6F6;
  border-color: #FFF6F6;
  border-width: thick;
  width: 380px;
}

.divider-sidebar {
  background: -webkit-gradient(linear, 0 0, 100 0, from(transparent), to(transparent), color-stop(50%, #FFF6F6));
  border-color: #FFF6F6;
  border-width: thick;
  width: 170px;
  margin: 0;
}

.btn-noglow {
  box-shadow: none !important;
  outline: 0px !important;
}

.btn-center {
  text-align: center;
}
```

```
.form {
  width: 35%;
}
```

Explanation

`.welcomeview`
- **Purpose**: Sets the height of elements with this class to 80% of the viewport height.
- **Property**: `height: 80vh;`

`.about-text`
- **Purpose**: Sets the text color of elements with this class to black.
- **Property**: `color: rgb(0, 0, 0);`

`.divider`
- **Purpose**: Styles elements as dividers with a specific background color, border color, border width, and width.
- **Properties**:
  - `background-color: #FFF6F6;`
  - `border-color: #FFF6F6;`
  - `border-width: thick;`
  - `width: 380px;`

`.divider-sidebar`
- **Purpose**: Styles elements as sidebar dividers with a gradient background, specific border color, border width, width, and no margin.
- **Properties**:
  - `background: -webkit-gradient(linear, 0 0, 100 0, from(transparent), to(transparent), color-stop(50%, #FFF6F6));`
  - `border-color: #FFF6F6;`
  - `border-width: thick;`
  - `width: 170px;`
  - `margin: 0;`

`.btn-noglow`
- **Purpose**: Removes the box shadow and outline from buttons to prevent glow effects.
- **Properties**:
  - `box-shadow: none !important;`
  - `outline: 0px !important;`

`.btn-center`
- **Purpose**: Centers the text within buttons.
- **Property**: `text-align: center;`

`.form`
- **Purpose**: Sets the width of form elements to 35%.
- **Property**: `width: 35%;`

Summary
This CSS code provides various classes for styling elements in your application. These classes include styles for setting heights, text colors, divider styles, button styles, and form widths. Each class is designed to apply specific styles to elements, ensuring a consistent and visually appealing layout.

Example Usage
Here is an example of how these CSS classes might be used in an HTML file:
```html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Styled Page</title>
 <link rel="stylesheet" href="App.css">
</head>
<body>
 <div class="welcomeview">
  <h1>Welcome to the Application</h1>
 </div>
 <div class="about-text">
  <p>This is some text about the application.</p>
 </div>
 <hr class="divider">
 <hr class="divider-sidebar">
 <button class="btn-noglow btn-center">Click Me</button>
 <form class="form">
  <input type="text" placeholder="Enter text">
 </form>
</body>
</html>
```
In this example:
- **App.css**: The CSS file containing the provided styles is linked in the HTML file.
- **welcomeview**: The `welcomeview` class is applied to a `div` element, setting its height to 80% of the viewport height.
- **about-text**: The `about-text` class is applied to a `div` element, setting the text color to black.

- **divider**: The `divider` class is applied to an `hr` element, styling it as a divider.
- **divider-sidebar**: The `divider-sidebar` class is applied to another `hr` element, styling it as a sidebar divider.
- **btn-noglow**: The `btn-noglow` class is applied to a `button` element, removing its box shadow and outline.
- **btn-center**: The `btn-center` class is also applied to the `button` element, centering its text.
- **form**: The `form` class is applied to a `form` element, setting its width to 35%.

This setup ensures that the elements are styled consistently and visually appealing, providing a user-friendly interface with responsive design.

Similar code found with 2 license types

The provided code snippet is from a

tsconfig.json

file, which is used to configure the TypeScript compiler options for a project. Here's a detailed breakdown of the relevant sections:

###

tsconfig.json

Structure
The

tsconfig.json

file is a JSON configuration file that specifies the root files and the compiler options required to compile a TypeScript project.

Relevant Sections

Compiler Options
```jsonc
// "noPropertyAccessFromIndexSignature": true,      /* Enforces using indexed accessors for keys declared using an indexed type. */
// "allowUnusedLabels": true,                  /* Disable error reporting for unused labels. */
// "allowUnreachableCode": true,                /* Disable error reporting for unreachable code. */
```

/* Completeness */
// "skipDefaultLibCheck": true,   /* Skip type checking .d.ts files that are included with TypeScript. */
"skipLibCheck": true   /* Skip type checking all .d.ts files. */
```
- **noPropertyAccessFromIndexSignature**: When enabled, this option enforces using indexed accessors for keys declared using an indexed type. It is currently commented out.
- **allowUnusedLabels**: When enabled, this option disables error reporting for unused labels. It is currently commented out.
- **allowUnreachableCode**: When enabled, this option disables error reporting for unreachable code. It is currently commented out.
- **skipDefaultLibCheck**: When enabled, this option skips type checking for `.d.ts` files that are included with TypeScript. It is currently commented out.
- **skipLibCheck**: When enabled, this option skips type checking for all `.d.ts` files. This can speed up the compilation process but may hide some type errors.

#### Include
```jsonc
"include": ["src"]
```
- **include**: Specifies an array of file glob patterns that should be included in the project. In this case, it includes all files in the `src` directory.

Summary
The

tsconfig.json

 file configures the TypeScript compiler options for your project. The provided snippet includes options for skipping type checks on declaration files and specifies the `src` directory as the root for included files. Some options are commented out, indicating they are not currently in use but can be enabled if needed.

Example

tsconfig.json

Here is an example of a complete

tsconfig.json

 file with the provided options:
```json

```
{
  "compilerOptions": {
    "target": "es5",                          /* Specify ECMAScript target version: 'ES3' (default), 'ES5', 'ES2015', 'ES2016', 'ES2017', 'ES2018', 'ES2019', 'ES2020', or 'ESNEXT'. */
    "module": "commonjs",                     /* Specify module code generation: 'none', 'commonjs', 'amd', 'system', 'umd', 'es2015', 'es2020', or 'ESNext'. */
    "strict": true,                           /* Enable all strict type-checking options. */
    "skipLibCheck": true                      /* Skip type checking all .d.ts files. */
  },
  "include": ["src"]
}
```

In this example:
- **compilerOptions**: Specifies various compiler options, including the target ECMAScript version, module system, strict type-checking, and skipping type checks for declaration files.
- **include**: Specifies the `src` directory as the root for included files.

This setup ensures that the TypeScript compiler is configured to compile the project efficiently while maintaining strict type-checking for the source files.

Similar code found with 1 license type

The provided

package.json

file is a configuration file for a Node.js project. It includes metadata about the project, scripts for running tasks, and lists of dependencies and devDependencies. Here's a detailed breakdown of the file:

Metadata
```json
{
  "name": "backend",
  "version": "1.0.0",
  "description": "backend code for to-do list app.",
  "main": "index.js",
  "keywords": [],
  "author": "Helena, Rui, Vasco",
  "license": "ISC",
```
- **name**: The name of the project (

backend

).
- **version**: The current version of the project (`1.0.0`).
- **description**: A brief description of the project (`backend code for to-do list app.`).
- **main**: The entry point of the application (`index.js`).
- **keywords**: An array of keywords related to the project (currently empty).
- **author**: The authors of the project (`Helena, Rui, Vasco`).
- **license**: The license for the project (`ISC`).

Scripts
```json
 "scripts": {
  "compile": "npx tsc && node ./build/main.js",
  "dev": "node ./node_modules/nodemon/bin/nodemon.js -e ts --exec \" npm run compile
\""
 },
```
- **scripts**: Defines a set of commands that can be run using `npm run <script-name>`.
 - **compile**: Compiles the TypeScript code and then runs the compiled JavaScript file (`npx tsc && node ./build/main.js`).
 - **dev**: Runs the application in development mode using `nodemon` to watch for changes in TypeScript files and recompile them (`node ./node_modules/nodemon/bin/nodemon.js -e ts --exec " npm run compile "`).

Dependencies
```json
 "dependencies": {
  "express": "^4.18.2",
  "nodemailer": "^6.9.16",
  "sqlite3": "^5.1.4"
 }
```
- **dependencies**: Lists the packages required for the application to run.
 - **express**: A web framework for Node.js (`^4.18.2`).
 - **nodemailer**: A module for sending emails (`^6.9.16`).
 - **sqlite3**: A library for interacting with SQLite databases (`^5.1.4`).

DevDependencies
```json
 "devDependencies": {
  "@types/express": "^4.17.14",
  "@types/nedb": "^1.8.12",
```

```
    "@types/node": "^18.11.9",
    "nodemon": "^2.0.22",
    "typescript": "^4.9.5"
  }
}
```
- **devDependencies**: Lists the packages required for development.
  - **@types/express**: TypeScript type definitions for Express (`^4.17.14`).
  - **@types/nedb**: TypeScript type definitions for NeDB (`^1.8.12`).
  - **@types/node**: TypeScript type definitions for Node.js (`^18.11.9`).
  - **nodemon**: A tool that helps develop Node.js applications by automatically restarting the application when file changes are detected (`^2.0.22`).
  - **typescript**: The TypeScript language (`^4.9.5`).

Summary
The

package.json

 file provides essential information about the project, including metadata, scripts for running tasks, and lists of dependencies and devDependencies. This setup ensures that the project can be easily managed, built, and run in both development and production environments.

Example Usage
Here is an example of how you might use the scripts defined in the

package.json

 file:

1. **Install Dependencies**:
   ```sh
   npm install
   ```

2. **Compile the TypeScript Code**:
   ```sh
   npm run compile
   ```

3. **Run the Application in Development Mode**:
   ```sh
   npm run dev
   ```

```

This setup provides a clear and structured way to manage the project's dependencies, build process, and development workflow.

The provided excerpt is from a

package-lock.json

file, which is automatically generated by npm to ensure consistent installations across different environments. This file locks the versions of the project's dependencies and their dependencies. Here's a detailed breakdown of the relevant sections:

###

package-lock.json

Structure
The

package-lock.json

file contains detailed information about the dependencies of a project, including their versions, resolved URLs, integrity hashes, and other metadata.

### Relevant Sections

#### `mkdirp`
```json
"bin": {
  "mkdirp": "bin/cmd.js"
},
"engines": {
  "node": ">=10"
}
```
- **bin**: Specifies the executable file for the `mkdirp` package.
  - **mkdirp**: The command that points to `bin/cmd.js`.
- **engines**: Specifies the Node.js version compatibility.
  - **node**: The minimum required Node.js version is 10.

`to-regex-range`
```json

"node_modules/to-regex-range": {
 "version": "5.0.1",
 "resolved": "https://registry.npmjs.org/to-regex-range/-/to-regex-range-5.0.1.tgz",
 "integrity": "sha512-
65P7iz6X5yEr1cwcgvQxbbIw7Uk3gOy5dIdtZ4rDveLqhrdJP+Li/Hx6tyK0NEb+2GCyneCMJiG
qrADCSNk8sQ==",
 "dev": true,
 "license": "MIT",
 "dependencies": {
  "is-number": "^7.0.0"
 },
 "engines": {
  "node": ">=8.0"
 }
}
```
- **version**: The version of the `to-regex-range` package (`5.0.1`).
- **resolved**: The URL where the package was resolved from.
- **integrity**: The integrity hash for verifying the package.
- **dev**: Indicates that this is a development dependency.
- **license**: The license type for the package (`MIT`).
- **dependencies**: Lists the dependencies required by this package.
 - **is-number**: A dependency with a version requirement of `^7.0.0`.
- **engines**: Specifies the Node.js version compatibility.
 - **node**: The minimum required Node.js version is 8.

`toidentifier`
```json
"node_modules/toidentifier": {
 "version": "1.0.1",
 "resolved": "https://registry.npmjs.org/toidentifier/-/toidentifier-1.0.1.tgz",
 "integrity": "sha512-
o5sSPKEkg/DIQNmH43V0/uerLrpzVedkUh8tGNvaeXpfpuwjKenlSox/2O/BTlZUtEe+JG7s5Yh
Ez608PlAHRA==",
 "license": "MIT",
 "engines": {
  "node": ">=0.6"
 }
}
```
- **version**: The version of the `toidentifier` package (`1.0.1`).
- **resolved**: The URL where the package was resolved from.
- **integrity**: The integrity hash for verifying the package.
- **license**: The license type for the package (`MIT`).
```

- **engines**: Specifies the Node.js version compatibility.
  - **node**: The minimum required Node.js version is 0.6.

`touch`
```json
"node_modules/touch": {
  "version": "3.1.0",
  "resolved": "https://registry.npmjs.org/touch/-/touch-3.1.0.tgz",
  "integrity": "sha512-
WBx8Uy5TLtOSRtIq+M03/sKDrXCLHxwDcquSP2c43Le03/9serjQBIztjRz6FkJez9D/hleyAXT
BGLwwZUw9lA==",
  "dev": true,
  "dependencies": {
    "nopt": "~1.0.10"
  },
  "bin": {
    "nodetouch": "bin/nodetouch.js"
  }
}
```
- **version**: The version of the `touch` package (`3.1.0`).
- **resolved**: The URL where the package was resolved from.
- **integrity**: The integrity hash for verifying the package.
- **dev**: Indicates that this is a development dependency.
- **dependencies**: Lists the dependencies required by this package.
  - **nopt**: A dependency with a version requirement of `~1.0.10`.
- **bin**: Specifies the executable file for the `touch` package.
  - **nodetouch**: The command that points to `bin/nodetouch.js`.

Summary
The

package-lock.json

 file ensures consistent installations by locking the versions of the project's dependencies and their dependencies. It includes detailed information such as versions, resolved URLs, integrity hashes, licenses, and Node.js version compatibility. This setup helps maintain a stable and predictable development environment.

Example Usage
Here is an example of how you might use the

package-lock.json

file in a project:

1. **Install Dependencies**:
   ```sh
   npm install
   ```

2. **Run Development Server**:
   ```sh
   npm run dev
   ```

3. **Compile TypeScript Code**:
   ```sh
   npm run compile
   ```

This setup ensures that the project's dependencies are installed consistently across different environments, providing a stable and predictable development workflow.

Similar code found with 3 license types

The provided code defines an interface for server information, reads a JSON file containing server information, and parses it into an object. Here's a detailed breakdown of the code:

Imports
```typescript
import path from "path";
import fs from "fs";
```
- **path**: A Node.js module for working with file and directory paths.
- **fs**: A Node.js module for interacting with the file system.

Interface Definition
```typescript
export interface IServerInfo {
  smtp: {
    host: string,
    port: number,
    auth: {
      user: string,
      pass: string
    }
```

```
  }
}
```
- **IServerInfo**: An interface that defines the structure of the server information object.
 - **smtp**: An object containing SMTP server information.
  - **host**: The SMTP server host (string).
  - **port**: The SMTP server port (number).
  - **auth**: An object containing authentication information.
   - **user**: The username for authentication (string).
   - **pass**: The password for authentication (string).

Exported Variable
```typescript
// Export serverInfo interface
export let serverInfo: IServerInfo;
```
- **serverInfo**: A variable of type

IServerInfo

 that will hold the parsed server information.

Reading and Parsing JSON File
```typescript
const rawInfo: string = fs.readFileSync(
  path.join(__dirname, "../server/serverInfo.json"), "utf-8");
serverInfo = JSON.parse(rawInfo); // string to object
```
- **rawInfo**: A string containing the raw contents of the

serverInfo.json

 file.
 - **fs.readFileSync**: Reads the file synchronously.
 - **path.join**: Joins the directory name (

__dirname

) with the relative path to the

serverInfo.json

 file.
 - **"utf-8"**: Specifies the encoding to read the file as a string.

- **serverInfo**: The parsed JSON object assigned to the

serverInfo

 variable.
 - **JSON.parse**: Converts the JSON string into an object.

Summary
The code defines an interface for server information, reads a JSON file containing the server information, and parses it into an object. This setup ensures that the server information is structured and accessible throughout the application.

Example

serverInfo.json

Here is an example of what the

serverInfo.json

 file might look like:
```json
{
  "smtp": {
    "host": "smtp.example.com",
    "port": 587,
    "auth": {
      "user": "user@example.com",
      "pass": "password123"
    }
  }
}
```
In this example:
- **smtp**: An object containing SMTP server information.
 - **host**: The SMTP server host (

smtp.example.com

).
 - **port**: The SMTP server port (`587`).
 - **auth**: An object containing authentication information.
  - **user**: The username for authentication (

user@example.com

).
   - **pass**: The password for authentication (`password123`).

Example Usage
Here is an example of how you might use the

serverInfo

 variable in another part of your application:
```typescript
import { serverInfo } from './serverInfo';

console.log(`SMTP Host: ${serverInfo.smtp.host}`);
console.log(`SMTP Port: ${serverInfo.smtp.port}`);
console.log(`SMTP User: ${serverInfo.smtp.auth.user}`);
```
In this example:
- **serverInfo**: The parsed server information is imported and used to log the SMTP server details to the console.

This setup provides a clear and structured way to manage and access server information in your application, ensuring that the information is easily accessible and correctly typed.

The provided code sets up an Express.js server with various API endpoints for managing users, tasks, categories, and lists. Here's a detailed explanation of each part of the code:

Imports
```typescript
import path from "path";
import express, { Express, NextFunction, Request, Response } from "express";

import * as Users from "./tables/Users";
import { IUser } from "./tables/Users";
import * as Tasks from "./tables/Tasks";
import { ITask } from "./tables/Tasks";
import * as Categories from "./tables/Categories";
import { ICategory } from "./tables/Categories";
import * as Lists from "./tables/Lists";
import { IList } from "./tables/Lists";
```

```
- **path**: A Node.js module for handling file paths.
- **express**: The Express.js framework for building web applications.
- **Users, Tasks, Categories, Lists**: Modules for handling database operations related to users, tasks, categories, and lists.
- **IUser, ITask, ICategory, IList**: TypeScript interfaces for defining the structure of user, task, category, and list objects.

### Express Application Setup
```typescript
const app: Express = express();
app.use(express.json());
app.use("/", express.static(path.join(__dirname, "../../client/dist")));
```
- **app**: An instance of the Express application.
- **express.json()**: Middleware to parse incoming JSON requests.
- **express.static()**: Middleware to serve static files from the `../../client/dist` directory.

CORS Middleware
```typescript
app.use(function (inRequest: Request, inResponse: Response, inNext: NextFunction) {
  inResponse.header("Access-Control-Allow-Origin", "*");
  inResponse.header("Access-Control-Allow-Methods", "GET,POST,DELETE,OPTIONS");
  inResponse.header("Access-Control-Allow-Headers", "Origin,X-Requested-
With,Content-Type,Accept");
  inNext();
});
```
- **CORS Middleware**: Allows cross-origin requests by setting appropriate headers.

User Endpoints
```typescript
app.post("/registerUser", async (inRequest: Request, inResponse: Response) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const user: IUser = await usersWorker.registerUser(inRequest.body);
    inResponse.send(user);
  } catch (inError) {
    console.log(inError);
    inResponse.send("Error");
  }
});

app.post("/updateUserData", async (inRequest: Request, inResponse: Response) => {
```
```

```
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const user = await usersWorker.updateUserData(inRequest.body);
    inResponse.send(user);
  } catch (inError) {
    console.log(inError);
    inResponse.send("Error");
  }
});

app.get("/loginUser/:data", async (req: Request, res: Response) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const user: IUser | null = await usersWorker.loginUser(req.params.data);
    res.send(user);
  } catch (inError) {
    res.send("Error");
  }
});

app.get("/getUserById/:id", async (req, res) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const user: IUser = await usersWorker.getUserById(req.params.id);
    res.send(user);
  } catch (inError) {
    res.send("Error");
  }
});

app.get("/getUserByName/:name", async (req, res) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const users: IUser[] = await usersWorker.getUserByName(req.params.name);
    res.send(users);
  } catch (inError) {
    res.send("Error");
  }
});

app.get("/getUserByEmail/:email", async (req, res) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    const users: IUser = await usersWorker.getUserByEmail(req.params.email);
```

```
    res.send(users);
  } catch (inError) {
    res.send("Error");
  }
});

app.delete("/deleteAccount/:id", async (req, res) => {
  try {
    const usersWorker: Users.Worker = new Users.Worker();
    await usersWorker.deleteUser(req.params.id);
    res.send();
  } catch (inError) {
    res.send("Error");
  }
});
```
- **User Endpoints**: Handle user-related operations such as registration, updating data, login, retrieval by ID, name, or email, and account deletion.

Task Endpoints
```typescript
app.get("/getTask/:taskID", async (req, res) => {
  try {
    const tasksWorker: Tasks.Worker = new Tasks.Worker();
    const userTasks: ITask = await tasksWorker.getTask(req.params.taskID);
    res.send(userTasks);
  } catch (err) {
    console.log(err);
    res.send("No task found with id: " + req.params.taskID);
  }
});

app.get("/getTasks/:listID", async (req, res) => {
  try {
    const tasksWorker: Tasks.Worker = new Tasks.Worker();
    const userTasks: ITask[] = await tasksWorker.getTasks(req.params.listID);
    res.send(userTasks);
  } catch (err) {
    console.log(err);
    res.send("Error");
  }
});

app.post("/createTask", async (req, res) => {
```

```
  try {
    const taskWorker: Tasks.Worker = new Tasks.Worker();
    const taskID: ITask = await taskWorker.createTask(req.body);
    res.send(taskID);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});

app.post("/updateTaskState", async (req, res) => {
  try {
    const taskWorker: Tasks.Worker = new Tasks.Worker();
    const task: ITask = await taskWorker.updateTaskState(req.body.taskID,
req.body.newState);
    res.send(task);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});

app.delete("/deleteTask/:taskID", async (req, res) => {
  try {
    const taskWorker: Tasks.Worker = new Tasks.Worker();
    await taskWorker.deleteTask(req.params.taskID);
    res.send(true);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});
```
- **Task Endpoints**: Handle task-related operations such as retrieval by task ID or list ID, creation, updating state, and deletion.

Category Endpoints
```typescript
app.get("/getCategories/:userID", async (req, res) => {
  try {
    const categoryWorker: Categories.Worker = new Categories.Worker();
    const categories: ICategory[] = await
categoryWorker.getCategories(req.params.userID);
    res.send(categories);
```

```
  } catch (e) {
    console.log(e);
    res.send(e);
  }
});

app.get("/getCategory/:id", async (req, res) => {
  try {
    const categoryWorker: Categories.Worker = new Categories.Worker();
    const category: ICategory = await categoryWorker.getCategory(req.params.id);
    res.send(category);
  } catch (e) {
    console.log(e);
    res.send(e);
  }
});

app.post("/createCategory", async (req, res) => {
  try {
    const categoryWorker: Categories.Worker = new Categories.Worker();
    const category: ICategory = await categoryWorker.createCategory(req.body);
    res.send(category);
  } catch (e) {
    console.log(e);
    res.send(e);
  }
});

app.delete("/deleteCategory/:categoryID", async (req, res) => {
  try {
    const categoryWorker: Categories.Worker = new Categories.Worker();
    await categoryWorker.deleteCategory(req.params.categoryID);
    res.send(true);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});

app.post("/updateCategory", async (req, res) => {
  try {
    const categoryWorker: Categories.Worker = new Categories.Worker();
    const category: ICategory = await categoryWorker.editCategory(req.body);
    res.send();
```

```
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});
```
- **Category Endpoints**: Handle category-related operations such as retrieval by user ID or category ID, creation, updating, and deletion.

List Endpoints
```typescript
app.post("/createList", async (req, res) => {
  try {
    const listWorker: Lists.Worker = new Lists.Worker();
    const list: IList = await listWorker.createList(req.body);
    res.send(list);
  } catch (e) {
    console.log(e);
    res.send("Error creating list.");
  }
});

app.post("/editList", async (req, res) => {
  try {
    const listWorker: Lists.Worker = new Lists.Worker();
    const list: IList = await listWorker.editList(req.body);
    res.send(list);
  } catch (e) {
    console.log(e);
    res.send("Error editing list.");
  }
});

app.delete("/deleteList/:listID", async (req, res) => {
  try {
    const listsWorker: Lists.Worker = new Lists.Worker();
    await listsWorker.deleteList(req.params.listID);
    res.send(true);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});
```

```
app.delete("/deleteLists/:categoryID", async (req, res) => {
  try {
    const listsWorker: Lists.Worker = new Lists.Worker();
    await listsWorker.deleteListsFromCat(req.params.categoryID);
    res.send(true);
  } catch (inError) {
    console.log(inError);
    res.send(inError);
  }
});

app.get("/getLists/:userID", async (req, res) => {
  try {
    const listsWorker: Lists.Worker = new Lists.Worker();
    const lists: IList[] = await listsWorker.getLists(req.params.userID);
    res.send(lists);
  } catch (e) {
    console.log(e);
    res.send(e);
  }
});

app.get("/getList/:listID", async (req, res) => {
  try {
    const listsWorker: Lists.Worker = new Lists.Worker();
    const list: IList = await listsWorker.getList(req.params.listID);
    res.send(list);
  } catch (e) {
    console.log(e);
    res.send(e);
  }
});
```
- **List Endpoints**: Handle list-related operations such as creation, editing, deletion by list ID or category ID, and retrieval by user ID or list ID.

Starting the Server
```typescript
app.listen(8080);
console.log("Running...");
```
- **app.listen(8080)**: Starts the Express server on port 8080.
- **console.log("Running...")**: Logs a message indicating that the server is running.

Summary
This code sets up an Express.js server with various API endpoints for managing users, tasks, categories, and lists. It includes middleware for parsing JSON requests, serving static files, and handling CORS. Each endpoint follows a consistent structure with `try-catch` blocks for error handling, ensuring that any issues are logged and communicated back to the client. The server listens on port 8080 and logs a message indicating that it is running.

Similar code found with 2 license types

The provided code defines a method named

updateUserData

within a class, which is responsible for updating user data in a database. This method takes an object of type

IUser

as a parameter and returns a promise that resolves to an updated

IUser

object. The method constructs an SQL query to update the user's name, email, and password in the `users` table based on the user's ID.

Initially, the method sets up an SQL query and parameters to update the user's name, email, and password. The password is hashed using the

cyrb53

function before being included in the parameters. However, if the user's password is `"null"`, the method adjusts the query and parameters to update only the name and email fields, leaving the password unchanged.

The method returns a promise that executes the SQL query using the

db.run

function. If an error occurs during the execution of the query, the promise is rejected with the error. If the query is successful, the promise resolves by calling the

getUserById

 method with the user's ID to retrieve the updated user data.

In the

main.ts

 file, the

updateUserData

 method is called within an asynchronous function, where the updated user data is awaited and assigned to the

user

 variable. This ensures that the user data is updated in the database and the updated information is retrieved and available for further processing or response.

Overall, the

updateUserData

 method provides a structured way to update user information in the database, handling both the case where the password is updated and the case where it remains unchanged. The use of promises ensures that the method can be used asynchronously, allowing for efficient and responsive handling of user data updates.